UNIVERSIDAD TECNOLÓGICA NACIONAL · FACULTAD REGIONAL MENDOZA

Cátedra de Inteligencia Artificial · Ciclo 2026

Asistente experto para la selección personalizada de notebooks

Informe ejecutivo final

Actividad PG3 · Unidad 4 (grupal): orquestación final, pruebas del prototipo y armado de carpeta

Grupo 2 · Comisión 5K09

Integrante	Submódulo a cargo
Miranda, Mauricio	SM2 · Rendimiento y capacidad (RAM y virtualización)
Olea, Facundo	SM1 · Interpretación del pedido
de Coninck, Ramiro	SM3 · Gráfica y movilidad
Luconi, Martina	SM4 · Integración, costo-beneficio y recomendación

Fecha de entrega: 13 de noviembre de 2026

Entregable: `Grupo_2_PG3_ProyectoFinal.zip`

Índice

1. Portada institucional **1**

2. Resumen ejecutivo **4**

3. Descripción del problema **5**

4. Arquitectura del sistema **6**

4.1 Principios de diseño 6

4.2 Componentes 6

4.3 Capas del backend 7

4.4 Flujo de una consulta 8

4.5 Contrato entre componentes 8

4.6 Base de conocimiento y fuentes de información 9

4.7 Interfaz y uso básico 9

4.8 Decisiones de diseño 9

5. Base de conocimiento en Neo4j **10**

Ontología 10

Conteos (grafo cargado) 10

Cómo se carga y de quién es cada archivo 10

5.1 SM1 · Interpretación del pedido 11

5.2 SM2 · Rendimiento y capacidad (RAM y virtualización) 11

5.3 SM3 · Gráfica y movilidad 11

5.4 SM4 · Integración, costo-beneficio y recomendación 12

6. Integración con el LLM **14**

Patrón sándwich 14

Prompt maestro 14

Modelo, temperatura y salida estructurada 14

Explicador y verificador 14

Qué hace el LLM y qué no 15

Acierto real del router 15

7. Casos de uso integrados **16**

8. Pruebas y resultados **17**

8.1 Resumen 17

8.2 E2E 17

8.3 Router 17

8.4 Explicador y verificador 17

8.5 Lo que no se verificó 18

9. Evaluación del sistema **19**

9.1 Fortalezas 19

9.2 Limitaciones 19

9.3 Situaciones problemáticas observadas 20

9.4 Posibles errores y qué los cubre 20

9.5 Cómo leer las cifras del router 21

9.6 Oportunidades de mejora 21

9.7 Lo que no está hecho o no se verificó 21

10. Trazabilidad del conocimiento **23**

10.1 Dónde queda cada eslabón 23

10.2 Ejemplo reconstruido: E2E-9 23
10.3 Cómo auditar una conclusión 24
10.4 Límites de la trazabilidad 25
11. Reflexiones finales **26**
11.1 Reflexión sobre la metodología 26
11.2 Conclusiones del grupo 26
12. Referencias y fuentes consultadas **28**
Material de la cátedra 28
Trabajos previos del grupo 28
Fuentes de conocimiento del dominio 28
Bibliografía técnica 28
Software utilizado 29
Anexo A · Registro de correcciones (consigna §3.1) **30**
Anexo B · Manual de uso **42**
Anexo C · Diagramas del sistema **52**
Anexo D · Casos de uso integrados (consigna §3.4) **55**
Anexo E · Pruebas funcionales (consigna §3.5) **58**

2. Resumen ejecutivo

El problema. Elegir una notebook obliga a traducir lo que el comprador necesita («programar, virtualizar y jugar a veces», «1000 USD») en requisitos de hardware y a compararlos contra fichas que suelen venir incompletas. Hoy lo resuelve un técnico por experiencia. Este trabajo vuelve ese criterio explícito, verificable y explicable para el **asesor de ventas**, que es quien usa el sistema.

La solución. Un único asistente con una sola entrada (API `POST /consultar` y consola) que responde en español. Cuatro submódulos comparten un grafo de conocimiento en Neo4j (343 nodos y 683 relaciones): SM1 interpreta el pedido, SM2 evalúa el riesgo de RAM y la virtualización, SM3 la gráfica y la movilidad, y SM4 recomienda con costo-beneficio. El razonamiento son 57 reglas (23 de ellas difusas, tipo Mamdani) escritas como consultas Cypher fijas.

Cómo funciona. El asesor escribe la consulta. Un router (LLM `qwen2.5:3b` local, temperatura 0) arma un plan JSON; Python lo valida; Neo4j ejecuta Cypher fijo de solo lectura; un explicador (LLM) redacta con esa evidencia y un verificador en Python revisa cifras, modelos y reglas antes de responder. Si no pasa, sale una plantilla fija.

Principios. Neo4j manda la verdad: el LLM interpreta y explica, pero no calcula, no decide ni escribe Cypher. La incertidumbre se conserva (rangos, `desconocida`, perfil indeterminado). Cada consulta deja una traza auditable.

Ejemplo real (E2E-1, req_01: 1000 USD, perfil dominante virtualización con grado 0,899). La A queda `Recomendada (Alta)` con grado 0,785, y la H y la B completan el podio por desempate de ampliabilidad. La X se ofrece aparte por pasarse del presupuesto. La E queda `Descartada (excluyente incumplido)` aunque puntúa ≈ 0,81, porque su CPU no soporta virtualización. La G queda `Indeterminada: completar informacion` y la M `Recomendada condicionada por falta de informacion` (grado [0,504 ; 0,826]).

Medida (corridas del 11/11/2026)	Resultado
Regresión del grafo (CP1 a CP11)	11 de 11
Pruebas automáticas (`unittest`)	171, con 2 salteadas (piden Ollama)
E2E (5 integrales y 2 controles)	7 de 7 aprueban; 4,3 a 7,1 s las que redacta el LLM (una máquina)
Router: intenciones / resultado final	28 de 31 (90 %) / 30 de 31 (97 %)
Respuestas con explicación que salieron de la plantilla fija	0 de 5

Lo que falta, dicho sin vueltas.

- Los parámetros difusos (centros, sigmas, mínimos de RAM) **no están validados por el técnico**: toda respuesta lo advierte.
- El asistente lee los resultados que el grafo ya calculó para 3 pedidos cargados: **no registra pedidos nuevos ni infiere en vivo**, y el nivel 2 (PG3-21 a 27) no está hecho.
- `qwen2.5:3b` redacta con torpezas que el verificador no siempre corta: en una respuesta (E2E-2) dio como riesgo de RAM de la G un número que era su adecuación global.
- Los tiempos de punta a punta se midieron en una sola máquina (en otra, el plan solo tardó de 10 a 50 s); el catálogo de prueba tiene 9 modelos.

Grupo 2 · Comisión 5K09 · UTN FRM · Cátedra de Inteligencia Artificial. Miranda, Mauricio · Olea, Facundo · de Coninck, Ramiro · Luconi, Martina · 13/11/2026.

3. Descripción del problema

Elegir una notebook es una decisión con muchas variables técnicas (CPU, RAM y su ampliabilidad, GPU, pantalla, peso, autonomía) que el comprador no sabe traducir a partir de lo que necesita. Un comprador dice «la uso para programar y virtualizar, y a veces juego» y «tengo 1000 USD»; un **asesor** de venta tiene que convertir eso en requisitos y compararlos contra un catálogo cuyas fichas suelen estar incompletas: muchas no informan si la RAM es ampliable, si la CPU soporta virtualización o cuánto dura la batería en uso real.

Usuario y beneficiario. El sistema lo usa el asesor; el beneficiario es el comprador. La explicación se escribe para el asesor, que la transmite.

Por qué hace falta un experto. La tarea combina conocimiento técnico (qué exige cada perfil de uso, qué riesgo implica quedarse corto de RAM), criterio para la incertidumbre («a veces», «liviana», un dato faltante) y políticas de decisión (un requisito excluyente no se compensa con puntaje). Un técnico lo resuelve por experiencia; el sistema lo hace explícito en reglas crisp, lógica difusa Mamdani y un grafo de conocimiento en Neo4j, con una traza de cada conclusión.

Alcance del prototipo.

- Recomienda hasta tres notebooks según el uso y el presupuesto, con alertas y una explicación trazable.
- Interpreta pedidos cargados (`req_01`, `req_02`, `req_03`) sobre un catálogo de prueba de 9 modelos con precios al 01/10/2026.
- Conserva la incertidumbre: un dato faltante se informa como rango o como «falta verificar», nunca se completa en silencio.
- **Fuera de alcance:** decidir la compra, precios en tiempo real, medir componentes, teclado, sonido y construcción, y procesadores ARM. El LLM interpreta la consulta y redacta; no decide ni calcula.

4. Arquitectura del sistema

La arquitectura completa está en `docs/ARQUITECTURA.md` (con su contrato en código, `asistente/contrato.py`). Esta sección resume lo que hace cada componente, cómo viaja una consulta y cómo se usa el sistema.

4.1 Principios de diseño

1. **Neo4j maneja la verdad.** Toda cifra, estado, regla y alerta sale del grafo. El LLM no calcula ni decide.
2. **El LLM interpreta y explica, en dos llamadas.** El router arma un plan JSON y el explicador redacta a partir de la evidencia. Nunca escribe Cypher.
3. **Las consultas son plantillas fijas**, parametrizadas y en sesión de solo lectura (`READ_ACCESS`). La única escritura prevista, `SM1.registrar_pedido` y `SM1.aclarar_pedido`, es de nivel 2 y **no está habilitada** (sección 9).
4. **Todo se verifica en código:** el plan, y después las cifras, modelos y reglas citados. Si la respuesta del LLM no pasa, se usa una plantilla fija.
5. **La incertidumbre se conserva:** rangos `{min, max}`, `desconocida` e `indeterminado` viajan intactos hasta la respuesta.
6. **Solo tecnologías vistas en la cátedra** (`docs/RESTRICCIONES.md`): Python, Neo4j, Cypher, Ollama, LangChain como cliente del LLM, FastAPI y `unittest`. Sin agentes, sin *tool calling*, sin embeddings.

4.2 Componentes

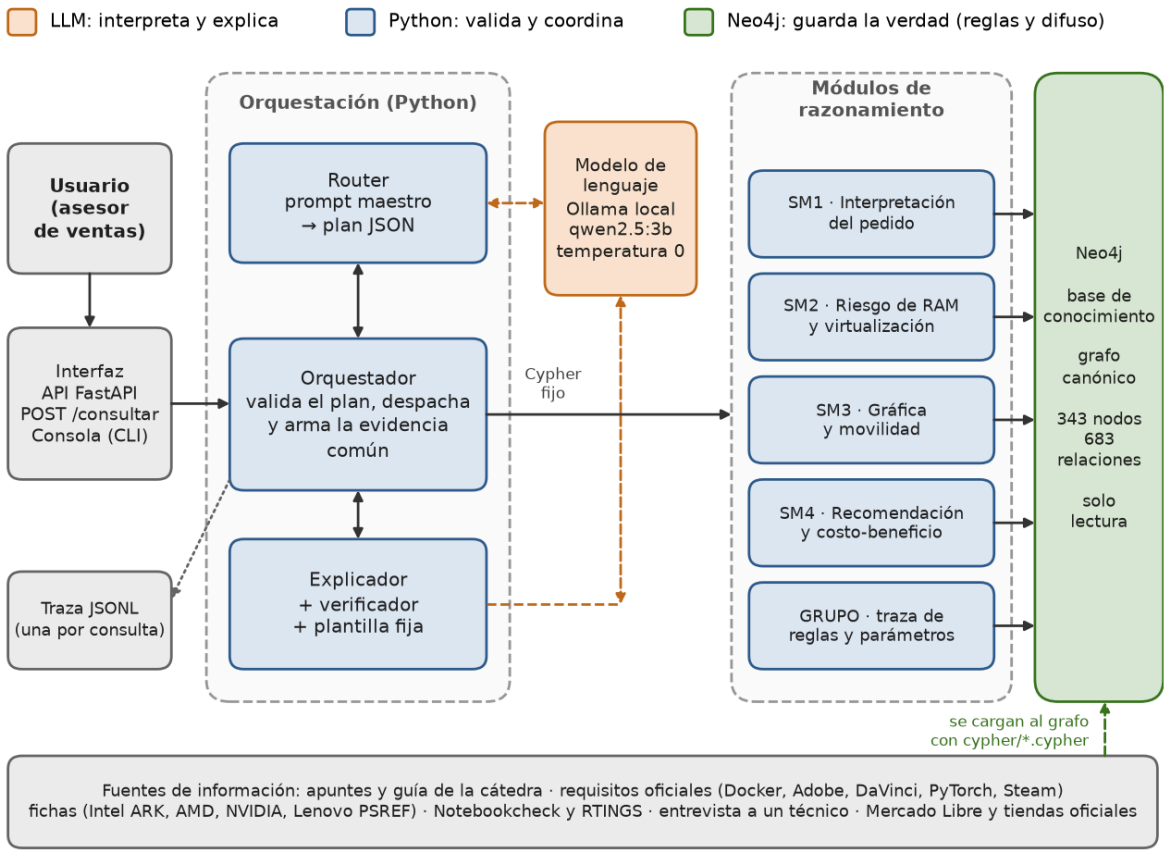


Figura 1 · Arquitectura del sistema. Los colores indican quién decide: el LLM interpreta y explica, Python valida y coordina, Neo4j guarda las reglas y el difuso.

Componente	Función	Archivo

Usuario	El asesor de ventas hace la consulta y recibe la explicación	—
Interfaz	Entrada única: `POST /consultar`, `GET /salud` y consola. Recibe, valida el formato y despacha; no tiene lógica ni Cypher	`asistente/api.py`, `asistente/cli.py`
Orquestador	Prevalida la entrada (vacía, sin letras, más de 1500 caracteres), pide el plan, ejecuta las consultas, arma la evidencia común, llama al explicador y registra la traza	`asistente/orquestador.py`
Router (prompt maestro)	Traduce la consulta a un plan JSON de intenciones cerradas y lo valida en Python: intención del conjunto, modelos del catálogo, fragmentos literales, pedido existente	`asistente/router.py`, `prompts/maestro.txt`
Modelo de lenguaje	`qwen2.5:3b` en Ollama local, temperatura 0, vía LangChain (`ChatPromptTemplate` y `ChatOllama` con salida estructurada)	`asistente/llm.py`
SM1 · Interpretación	Pedido interpretado: perfiles con grado, perfil dominante, intensidad gráfica, presupuesto y preguntas pendientes	`asistente/submodulos/sm1/`
SM2 · Rendimiento	Riesgo de RAM (difuso, FZ-R1 a FZ-R5) y virtualización como requisito excluyente	`asistente/submodulos/sm2/`
SM3 · Gráfica y movilidad	Búsqueda por gráfica, movilidad o ambas, y ficha de un modelo	`asistente/submodulos/sm3/`
SM4 · Recomendación	Ranking, explicación por notebook, comparación y costo-beneficio	`asistente/submodulos/sm4/`
GRUPO	Traza de reglas activadas (con su origen) y parámetros sin validar	`asistente/submodulos/grupo/`
Explicador	Un único prompt de explicación en voseo, un verificador común (controles a a h) y la plantilla fija	`asistente/explicador.py`, `prompts/explicacion.txt`
Base de conocimiento	Grafo canónico de PG2, cargado desde `cypher/*.cypher`	`cypher/`, `scripts/cargar_grafo.py`
Traza	Una línea JSON por consulta, con plan, Cypher, filas, evidencia, intentos y tiempos	`asistente/traza.py`

Las figuras de esta sección son versiones simplificadas, pensadas para leerse en una página vertical. Los diagramas completos de la tarea PG3-10, en hoja A4 apaisada y editables en Draw.io (arquitectura con los siete elementos de la consigna §3.3, secuencia de una consulta y ontología del grafo), están en el Anexo C y en `docs/diagramas/`. Los diagramas en formato Mermaid están además en `docs/ARQUITECTURA.md`.

4.3 Capas del backend

El backend sigue un esquema **controlador, servicio y repositorio**, con los esquemas (la forma de los datos) aparte. Cada submódulo repite la misma estructura en cuatro archivos.

Capa	En el sistema	Dentro de cada submódulo	Regla
Controlador	`api.py`, `cli.py`	`entrada.py`	Recibe, valida y despacha; sin lógica ni Cypher
Servicio	`orquestador.py`, `router.py`, `explicador.py`	`servicio.py`	Arma la evidencia (redondeo, rangos, literales); no recalcula ni decide
Repositorio	`grafo.py`, `llm.py`	`repositorio.py`	Acceso a datos: el Cypher fijo, donde viven las reglas y el difuso
Esquemas	`contrato.py`, modelos Pydantic de la API	`esquemas.py`	Forma de parámetros, evidencia y respuesta

Una capa solo llama a la de abajo, un submódulo **no importa a otro** (se hablan a través del grafo, donde cada evaluación es un nodo `Evaluacion`) y **no hay endpoints por submódulo**: toda consulta pasa por el router, la traza y el explicador.

4.4 Flujo de una consulta

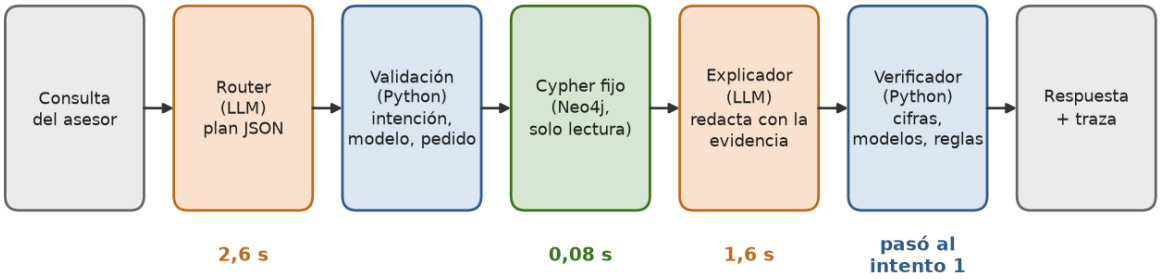


Figura 2 · Flujo de una consulta con los tiempos reales del E2E-9 (plan 2604 ms, consultas 80 ms, explicación 1647 ms).

1. **Prevalidación.** Una entrada vacía, sin letras o de más de 1500 caracteres se contesta sin llamar al LLM (`origen_respuesta = "validacion"`).
2. **Plan.** El router le pide al LLM un JSON con hasta tres intenciones del conjunto cerrado. Python lo valida y descarta lo que no está en el catálogo o no aparece literal en el texto. Si falta el pedido, pregunta «¿Sobre qué pedido?»: nunca asume `req_01`.
3. **Consultas.** Cada intención se despacha a su submódulo, que ejecuta Cypher fijo y devuelve `{"consultas", "evidencia"}`. El orquestador agrega siempre `SM1.interpretacion_pedido` cuando hay pedido.
4. **Explicación.** El LLM redacta con la evidencia común. El verificador revisa la respuesta; si falla, se regenera hasta dos veces y después sale la plantilla fija.
5. **Respuesta y traza.** Se devuelve `{respuesta, requerimiento_id, submodulos_consultados, origen_respuesta, intento, traza}` y se guarda la traza.

Caminos cortos, sin explicador: la entrada inválida; `fuera_de_alcance`, que devuelve un mensaje fijo sin tocar el grafo (0,69 s en el E2E-8); un plan con aclaración; y un error de Neo4j u Ollama, que devuelve `origen_respuesta = "error"` con un mensaje fijo.

4.5 Contrato entre componentes

Cada submódulo expone **una sola función**: `ejecutar(intencion, parametros, grafo) -> {"consultas", "evidencia"}`. El router solo puede pedir estas intenciones:

Submódulo	Intención	Qué responde
SM1	`interpretacion_pedido`	Qué pidió el comprador y cómo lo interpretó el sistema
SM2	`riesgo_ram`	Riesgo de quedarse corto de RAM por notebook, como rango si falta la ampliabilidad
SM2	`virtualizacion`	Si la CPU cumple el requisito excluyente
SM3	`buscar`	Búsqueda ordenada por gráfica, movilidad o mixta, con filtros opcionales
SM3	`ficha`	Ficha gráfica y de movilidad de un modelo
SM4	`recomendar`	Ranking con estado, grado y alternativa fuera de presupuesto
SM4	`explicar_notebook`	Lo que cumple, lo que no cumple, lo que falta verificar y las alertas de una notebook
SM4	`comparar`	De dos a tres notebooks lado a lado
SM4	`costo_beneficio`	Índice costo-beneficio de las recomendadas
GRUPO	`traza_reglas`	Reglas activadas para una notebook, con su fuerza y su origen
GRUPO	`parametros`	Parámetros sin validar

El contrato define 13 intenciones, pero solo 11 están habilitadas: `SM1.registrar_pedido` y `SM1.aclarar_pedido` son de nivel 2 y el router las rechaza.

La **evidencia** es lo único que ve el explicador. Sus reglas: los números salen del grafo con tres decimales; los rangos viajan como `{"min", "max"}`, nunca como un solo número; `desconocida`, `indeterminado` y los estados llevan los literales de D-06 sin tildes; cada regla trae su fuerza y su estado de validación; y cada alerta trae la notebook a la que pertenece, porque la atribución la controla el verificador.

4.6 Base de conocimiento y fuentes de información

El grafo (Neo4j 5.26) tiene 343 nodos y 683 relaciones, con 28 labels y 29 tipos de relación. Se arma al cargar los archivos `cypher/NN_*.cypher`, uno por dueño, que reproducen el canónico de PG2 corregido. El detalle de la ontología está en la sección 5, y su diagrama, en el Anexo C.

Las **fuentes de información** no se consultan en tiempo de ejecución: se volcaron al grafo al construirlo, como parámetros, etiquetas difusas, datos de fichas y precios. Son las que registró el grupo en PG1: apuntes y guía de la cátedra; requisitos oficiales (Docker, Adobe, DaVinci, PyTorch, Steam); fichas de fabricantes (Intel ARK, AMD, NVIDIA, Lenovo PSREF); Notebookcheck y RTINGS; una entrevista a un técnico; y precios de Mercado Libre y tiendas oficiales al 01/10/2026. El patrón es **Graph RAG sin embeddings**: la evidencia son las filas que devuelve el Cypher fijo, y el explicador cita reglas y nodos de ahí.

4.7 Interfaz y uso básico

El manual de uso completo está en el Anexo B (`docs/MANUAL_DE_USO.md`): instalación paso a paso en Windows, consultas de ejemplo, cómo leer una respuesta y errores comunes. Lo mínimo para usar el sistema:

Requisitos. Python 3, Neo4j 5.x, Ollama con el modelo `qwen2.5:3b` (`ollama serve`) y unos 4 GB de RAM libres para el modelo. Se copia `.env.example` a `.env`, se completa `NEO4J_PASSWORD` (nunca se versiona) y se exportan las variables antes de lanzar la consola o la API.

```
pip install -r requirements.txt
python -m scripts.cargar_grafo        # carga el grafo desde cypher/ (BORRA la base de ese Neo4j)
python -m scripts.verificar_grafo     # regresión CP1 a CP11
python -m asistente.cli "¿Qué me recomendás para req_01?"   # una consulta por consola
python -m asistente.cli --traza --pedido req_01              # modo interactivo, con la traza
uvicorn asistente.api:app             # API; documentación OpenAPI en /docs
python -m unittest                    # pruebas (con Neo4j y Ollama también corren las de integración)
```

`POST /consultar` recibe `{"texto": "...", "requerimiento_id": "req_01"}` (el pedido es opcional si el texto lo nombra) y devuelve la respuesta, el pedido, los submódulos consultados, el `origen_respuesta` (`llm`, `plantilla_fija`, `validacion`, `fuera_de_alcance` o `error`) y la traza completa. `GET /salud` informa el estado de Neo4j y de Ollama. Las preguntas que el sistema sabe responder son las 11 intenciones de la sección 4.5, sobre los pedidos cargados `req_01`, `req_02` y `req_03`.

4.8 Decisiones de diseño

- **Router con intenciones cerradas en lugar de Text-to-Cypher.** Dejar que un modelo de 3 mil millones de parámetros escriba Cypher habría roto «Neo4j maneja la verdad» y la garantía de solo lectura. El modelo elige entre 11 intenciones y Python valida.
- **Un verificador común en lugar de uno por submódulo.** Recorre la evidencia sin conocer la forma de cada submódulo, así que sumar uno nuevo no lo rompe.
- **Un contrato fijo y un submódulo por persona.** Permitió que cuatro integrantes trabajaran en paralelo; cambiarlo exige un PR aprobado por Miranda.
- **LangChain solo como cliente.** `ChatPromptTemplate`, `ChatOllama` y `RunnableLambda` para despachar; sin agentes. Es la única excepción registrada en `docs/RESTRICCIONES.md` §8 (el paquete `langchain-ollama`).

5. Base de conocimiento en Neo4j

Fuente: el grafo canónico de PG2 corregido (`docs/cypher/Grupo_2_PG2_U1_corregido.cypher`, decisión D-01), dividido en PG3 en un archivo por dueño sin cambiar su contenido.

Ontología

Capa	Labels	Para qué
Esquema	`Frame`	Una clase por concepto de la red semántica de PG1
Conocimiento	`Perfil`, `Requisito`, `VariableDifusa`, `EtiquetaDifusa`, `Regla` (`ReglaDifusa`, `ReglaCrisp`), `Parametro`, `ExpresionFrecuencia`, `GamaGPU`, `ConsultaExperto`	Perfiles de uso y sus requisitos, 15 variables y 47 etiquetas difusas, 57 reglas, parámetros, léxico de frecuencia y agenda de validación con el técnico
Instancias	`Comprador`, `Requerimiento`, `Actividad`, `Juego`, `Notebook`, `CPU`, `RAM`, `Almacenamiento`, `GPU`, `Pantalla`, `Bateria`	Pedidos y catálogo
Inferencia	`Evaluacion`, `Lectura`, `Alerta`, `Pregunta`, `Recomendacion`	Resultados por pedido × notebook, escenarios peor/mejor con dato faltante, alertas y recomendación

Relaciones principales: `FORMULA`, `INCLUYE`, `PERTENECE`, `TIENE_PERFIL_DOMINANTE`, `EXIGE`, `POSEE`, `EVALUA` y `SOBRE` (pedido → evaluación → notebook), `TIENE_LECTURA`, `PERTENECE_A` (pertenencia difusa), `ACTIVA {fuerza}` (traza de cada regla), `GENERA` (alertas), `REQUIERE_ACLARACION`, `PROPONE` y `PROPONE_FUERA_DE_PRESUPUESTO`.

Todo resultado que depende del pedido vive en la `(:Evaluacion)` de ese pedido × notebook (corrección de PG2): dos compradores pueden evaluar la misma notebook sin pisarse (caso CP11, Notebook_K en `req_01` y `req_03`).

Conteos (grafo cargado)

343 nodos y 683 relaciones. La carga dividida da **los mismos conteos** por label (28) y por tipo de relación (29) que el canónico entero: 57 de 57 iguales.

Cómo se carga y de quién es cada archivo

`python -m scripts.cargar_grafo` ejecuta `cypher/NN_*.cypher` en orden numérico, sentencia por sentencia; si una falla, informa el archivo y la línea. `python -m scripts.verificar_grafo` corre la regresión CP1–CP11 contra los esperados de PG1/PG2 §9.4: **11 de 11 OK**.

Archivo	Contenido	Dueño
`00_esquema.cypher`	Limpieza, índices y frames	Miranda
`01_conocimiento.cypher`	Perfiles, variables difusas, reglas, parámetros, léxico y gamas	Miranda
`02_instancias.cypher`	Compradores, pedidos y catálogo	Miranda
`10_sm1.cypher`	Fuzificación e inferencia de SM1	Olea
`15_evaluaciones.cypher`	Una `Evaluacion` por pedido × notebook	Miranda
`20_sm2.cypher`	Riesgo de obsolescencia de la RAM	Miranda
`30_sm3.cypher`	Movilidad y grado de GPU	de Coninck
`40_sm4.cypher`	Filtro de excluyentes y recomendación	Luconi
`90_verificacion.cypher`	Consultas C1–C3 y CP1–CP11 (no se cargan)	Miranda

Arreglos de D-06 aplicados en PG3: los parámetros, el léxico y las gamas pasan de `'provisional'` a `'sin validar'` (hoy no queda ningún nodo `'provisional'`), y la prioridad `grafica` suma su incremento al peso gráfico (REC-R5), como dice el enunciado de la regla.

5.1 SM1 · Interpretación del pedido

Qué resuelve. Convierte el pedido en lenguaje natural en un requerimiento formal: perfiles de uso con su grado, perfil dominante, intensidad gráfica, presupuesto y prioridad. Es la clave `pedido` de la evidencia común, que acompaña **todas** las respuestas con pedido.

Léxico de frecuencia. Las expresiones del comprador («todos los días», «a veces», «los fines de semana») son nodos `(:ExpresionFrecuencia {texto, horas_ref})`: se traducen a horas semanales de referencia (`dedicacion_h`), sin que el LLM invente números.

Reglas INT-RD1..RD7 (Mamdani, sin validar).

- INT-RD1..RD3 dan el `peso_perfil` de cada actividad a partir de la dedicación: rara vez u ocasional → marginal; frecuente → relevante; principal → determinante.
- INT-RD4..RD7 dan la `intensidad_grafica` cruzando la exigencia del juego con la dedicación (por ejemplo, INT-RD4: exigencia alta y dedicación frecuente o principal → intensidad alta).
- Un perfil impone requisitos si su grado supera el umbral de 0,4. El dominante es el de mayor grado (INT-R11; INT-R12 marca empates).

Qué devuelve (`SM1.interpretacion_pedido`, consulta L1). Para `req_01`: VIRT dominante con grado 0,899, que impone requisitos; GAM 0,313 como preferencia; intensidad baja; 1000 USD; prioridad rendimiento. Para `req_02` («Quiero una notebook para la facu, tengo 600 USD»), el perfil queda `indeterminado` con la pregunta pendiente y **sin asumir OFI** (CP10, INT-R13). Un perfil cuyo grado sea un rango se informa como `{min, max}` y, si es el dominante, como provisional (D-09).

5.2 SM2 · Rendimiento y capacidad (RAM y virtualización)

Qué resuelve. Evalúa, para cada pedido × notebook, el riesgo de que la RAM quede corta para el perfil y si la CPU cumple el requisito excluyente de virtualización.

Modelo difuso (FZ-R1..R5, Mamdani). Entradas: la suficiencia ρ = RAM ÷ mínimo del perfil (insuficiente, justa, suficiente, holgada) y el índice de ampliabilidad a (poca, media, mucha), que combina ranuras libres y margen hasta la RAM máxima. Reglas: insuficiente y poca → riesgo alto (FZ-R1); insuficiente y media o mucha → medio (FZ-R2); justa y poca → medio (FZ-R3); justa y media o mucha → bajo (FZ-R4); suficiente u holgada → bajo (FZ-R5). AND = MIN, OR = MAX, agregación MAX y centroide sobre 0–100.

Dato faltante. Si la ficha no informa la ampliabilidad (REN-R10), se evalúan dos escenarios (a = 0 y a = 1, FZ-R6) y el riesgo se informa como **rango**, nunca como promedio. Si el peor caso es alto, se genera la alerta FZ-R7.

Qué devuelve.

- `SM2.riesgo_ram` (consulta C2): por notebook, la RAM, su tipo, ρ, el índice a (o `desconocida`), el riesgo (número o `{min, max}`), su etiqueta, las reglas activadas con su fuerza y las alertas con su notebook. Ejemplo: Notebook_G en `req_01` da riesgo [49,58 ; 81,46], RAM `desconocida` y las alertas REN-R10 y FZ-R7. Notebook_K da 49,55, riesgo medio.
- `SM2.virtualizacion` (consulta C1): por notebook, el resultado (`cumple`, `no cumple`, `falta informacion`) y su estado. Notebook_E no soporta virtualización y queda `Descartada (excluyente incumplido)` aunque su adecuación sea alta (REN-R1 + REC-R3).

Parámetros sin validar. Los centros y sigmas de las etiquetas, y los mínimos de RAM por perfil, están `'sin validar'` por el técnico: toda respuesta lleva la advertencia correspondiente.

5.3 SM3 · Gráfica y movilidad

Qué resuelve. Evalúa la adecuación gráfica (GPU y pantalla) para el perfil del pedido y la movilidad (peso y autonomía) de cada notebook, y permite buscar y ordenar el catálogo por esos criterios.

Funciones de pertenencia (gaussianas, sin validar). Peso: liviana (centro 1,2 kg), intermedia (1,8), pesada (2,5). Autonomía: baja (3 h), media (6,5), alta (10). Potencia relativa de la GPU (TGP ÷ TGP máximo): restringida (0,35), moderada (0,60), plena (0,80).

Reglas GM-RD1..RD6 (Mamdani).

- Movilidad: GM-RD1 liviana y autonomía alta → alta; GM-RD2 combinaciones intermedias → media; GM-RD3 pesada o autonomía baja → baja.

- Grado de GPU: cruza el margen entre la gama de la GPU y la que exige el perfil (sobra, justa, insuficiente) con la potencia relativa (GM-RD4 → pleno, GM-RD5 → parcial, GM-RD6 → insuficiente).
- GM-R12: si no hay autonomía medida se usa el 70 % de la declarada (dato estimado); si no hay ninguna, la movilidad no se evalúa.

Qué devuelve.

- `SM3.buscar` (consulta unificada, hasta 5 resultados): ordena por gráfica, movilidad o mixta, con filtros opcionales (GPU dedicada, gama mínima, peso máximo, batería mínima, sRGB mínimo). Solo trae las evaluaciones del pedido consultado. Con `req_01` y GPU dedicada, Notebook_X aparece con grado de GPU 0,74.
- `SM3.ficha`: la ficha gráfica y de movilidad de un modelo. Notebook_M no informa autonomía: su movilidad queda `null` y se informa como dato faltante, sin inventar un número.
- Los filtros imposibles (por ejemplo, un peso máximo de 50 kg) se rechazan con una aclaración antes de consultar el grafo.

5.4 SM4 · Integración, costo-beneficio y recomendación

Qué resuelve. Cierra el razonamiento: con la adecuación de rendimiento (SM2) y de gráfica y movilidad (SM3) decide qué notebooks se recomiendan, en qué orden y con qué alertas. El LLM no calcula nada: lee del grafo el estado, el grado y el ranking que ya infirió el bloque 10.f.

Reglas REC-R1..R15 (todas `'sin validar'`).

Regla	Qué hace	Estado
REC-R1, R2, R6, R12	Cortes crisp de PI1 (presupuesto, alternativa fuera de presupuesto, umbral, varias fuera de presupuesto)	Reemplazadas por REC-RD1..RD5
REC-R3	Filtro de excluyentes	Vigente
REC-R4, R5	Pesos por perfil y ajuste por prioridad	Vigentes
REC-R7, R13	Alertas de sobredimensionamiento e infradimensionamiento (RAM)	Vigentes
REC-R8	Desempate por ampliabilidad	Vigente
REC-R9	Índice costo-beneficio	Vigente
REC-R10	«Sin alternativa adecuada»	Vigente
REC-R11	Explicación en tres bloques (cumple, no cumple, falta verificar)	Vigente: la arma `SM4.explicar_notebook`; la verifica el explicador (PG3-13)
REC-R14	RAM soldada por debajo de lo recomendado	Pasó a SM2 (FZ-R3 y FZ-R7)
REC-R15	Dato faltante que solo ordena: rango mín./máx.	Vigente
REC-RD1..RD5	Grado de recomendación (Mamdani) a partir de la adecuación global y del excedente de presupuesto	Vigentes

Filtro de excluyentes (REC-R3 + REN-R1). Si el perfil exige virtualización y la CPU no la soporta, la notebook queda `Descartada (excluyente incumplido)` sin importar su puntaje. Si la ficha no lo informa queda `Indeterminada: completar informacion` y no se compara. Notebook_E tiene una adecuación de 0,81, casi igual a la de B (0,813) y H (0,811), y aun así queda descartada.

Puntaje con pesos por perfil. La adecuación global es `peso_rendimiento · rendimiento + peso_gráfica · gráfica + peso_movilidad · movilidad` (REC-R4). Los pesos base salen del perfil dominante: OFI 0,4 / 0,1 / 0,5, DEV 0,6 / 0,1 / 0,3, VIRT 0,7 / 0,1 / 0,2, DIS 0,4 / 0,5 / 0,1, VID3D 0,5 / 0,4 / 0,1, GAM 0,3 / 0,6 / 0,1 y DATOS 0,7 / 0,1 / 0,2. La prioridad declarada suma 0,2 a su peso y se renormaliza (REC-R5): VIRT con prioridad rendimiento da 0,75 / 0,083 / 0,167. Después REC-RD1..RD5 cruzan esa adecuación con el excedente de presupuesto y dan el grado (A 0,785 es `Recomendada (Alta)`; B, H y X son `Recomendada (Media)`). Si falta la movilidad (Notebook_M no informa la autonomía) la adecuación y el grado quedan como rango, [0,741 ; 0,908] y [0,504 ; 0,826], y el estado es `Recomendada condicionada por falta de informacion`: no se completa ni se promedia.

Desempate (REC-R8). Entran hasta 3 notebooks dentro del presupuesto. Dos notebooks cuyo grado cae en la misma franja de 0,05 (`P_MARGEN_EMPATE`) empatan, y gana la de mayor ampliabilidad: H (0,626,

ampliabilidad alta) queda segunda y B (0,632, baja) tercera, aunque B rinde más por dólar (índice 1,161 contra 0,811). Con rango se ordena por el peor caso, así que M no entra. X, que se pasa 5 % del presupuesto, se ofrece aparte como alternativa fuera de presupuesto.

Qué devuelve.

- `SM4.recomendar` (Q1, Q2, Q3 y Q6B): el ranking con puesto, estado, grado (o `{min, max}`), precio, ampliabilidad y con quién empata; la alternativa fuera de presupuesto; las no propuestas con su motivo y su regla (E: REC-R3; C y K: REC-RD5; G: REC-R3; M: fuera de los 3 primeros puestos); y el criterio de desempate.
- `SM4.explicar_notebook` (Q4, la L2 del bloque 13): una notebook en un pedido, en los bloques cumple, no cumple, falta verificar y alertas. Para M: grado [0,504 ; 0,826] y «falta verificar» la autonomía.
- `SM4.comparar` (Q6 y Q6B): de 2 a 3 notebooks lado a lado, con el motivo del excluyente.
- `SM4.costo_beneficio` (Q7): las notebooks recomendadas de mayor a menor índice. Para M el índice se calcula con el peor caso y la evidencia lo marca.
- Un pedido o un modelo sin evaluación se rechaza con una aclaración: no se devuelve una lista vacía.

6. Integración con el LLM

Patrón sándwich

El LLM aparece en los dos extremos del flujo, y el conocimiento ocupa el medio:

1. **Router (LLM):** traduce la consulta a un plan JSON de intenciones cerradas.
2. **Validación (Python):** descarta intenciones, modelos y pedidos inválidos antes de tocar el grafo.
3. **Cypher fijo (Neo4j):** cada intención ejecuta consultas escritas de antemano, en modo solo lectura. El LLM nunca escribe Cypher.
4. **Explicador (LLM) y verificador (Python):** redactan la respuesta solo con la evidencia que devolvió el grafo. Si una cifra, un modelo o una regla no está en la evidencia, se regenera hasta dos veces y después se usa una plantilla fija.

Prompt maestro

`prompts/maestro.txt` aplica las técnicas de A10:

- un **rol** acotado («sos el router; no respondas la pregunta»);
- **límites**: no calcula, no inventa modelos, cifras ni pedidos, y nunca supone `req_01`;
- las **13 intenciones**, una línea cada una, con cuándo usarla;
- **restricciones negativas** y un **formato cerrado**;
- **9 ejemplos**, uno de ellos multisubmódulo.

En cada llamada recibe el catálogo, los pedidos cargados y el pedido de la sesión. El plan admite hasta 3 consultas, de modo que una pregunta como «¿Por qué la G quedó indeterminada y qué riesgo de RAM tiene en req_01?» se responde con SM4 y SM2 a la vez.

Modelo, temperatura y salida estructurada

- **Modelo:** `qwen2.5:3b` en Ollama local, sin costo de API y con unos 4 GB de RAM, por lo que corre en las cuatro notebooks del grupo (D-05).
- **Temperatura 0**, para que las respuestas sean reproducibles.
- **LangChain como cliente**, siguiendo el ejemplo de PG3 de la cátedra: `ChatPromptTemplate | ChatOllama.with_structured_output(esquema)`. El esquema JSON del plan, con un `enum` de intenciones y `maxItems: 3`, obliga al modelo a devolver un JSON válido, que compensa la debilidad de un modelo chico.
- **Despacho:** cada intención validada va a su submódulo mediante un mapa de `RunnableLambda`.
- **Sin agentes ni *tool calling*:** qué consulta se ejecuta lo decide el código, no el modelo.

Explicador y verificador

`asistente/explicador.py` y `prompts/explicacion.txt` son comunes a todos los submódulos. El LLM recibe la evidencia común **compactada** (reglas repetidas juntas, listas largas cortadas a 12 filas, porque `qwen2.5:3b` tiene poco contexto) y redacta en voseo para el asesor, con un máximo de 150 palabras en el prompt. El verificador compara la respuesta contra la evidencia **original**, sin compactar, con ocho controles:

Control	Qué exige
a	Cada cifra está en la evidencia (con redondeo y coma o punto) y, si la oración habla de una sola notebook, es un dato de esa notebook o del pedido
b	Cada `Notebook_X` y cada regla citada existen
c	Cada alerta está en la notebook que la tiene
d	Hasta 180 palabras y no vacía
e	Está la advertencia de parámetros sin validar, literal (si el modelo no la copia, la agrega el código)
f	Un rango no se da como un valor intermedio ni con un solo extremo
g	Si el pedido es `indeterminado`, no se nombra un perfil

h	Los estados, comparados sin tildes, son los de la evidencia

Si algún control falla, se vuelve a pedir la respuesta pasándole al modelo los problemas y su respuesta anterior, hasta `MAX_REGENERACIONES = 2` veces. Si sigue fallando, sale la **plantilla fija**, armada en código solo con la evidencia (`origen_respuesta = "plantilla_fija"`). En los 5 E2E con explicación (11/11/2026) el verificador aprobó a la primera: 0 de 5 salieron de la plantilla. El verificador corta lo inventado, pero no garantiza que la redacción sea buena ni que cada cifra lleve el nombre correcto (sección 9).

Qué hace el LLM y qué no

Hace	No hace
Interpretar la consulta y elegir intenciones	Calcular grados, riesgos o puntajes
Extraer modelos, pedidos y filtros mencionados	Decidir si una notebook se recomienda o se descarta
Redactar la explicación para el asesor	Escribir o elegir Cypher
	Completar un pedido o un dato que el usuario no dio

Acierto real del router

El router se probó con `tests/datos/consultas_router.csv`, con `qwen2.5:3b` a temperatura 0. La medición final (11/11/2026) usa **31 consultas**: las 25 de la primera medición, que cubren las 13 intenciones, 4 multisubmódulo, 3 fuera de dominio, 2 sin pedido, 1 con pedido de sesión y 2 con modelos inexistentes, más las 6 consultas de los E2E.

Medida (31 consultas)	Resultado
Intenciones correctas en el plan del LLM	**28 / 31 (90 %)**
Resultado final después de la validación en Python (plan, aclaración o fuera de alcance)	**30 / 31 (97 %)**

Las tres intenciones que fallan son R13 (responder una pregunta pendiente, que es de nivel 2 y el router rechaza a propósito) y R26 y R30, donde el plan omite una de las intenciones esperadas. Los detalles están en la sección 8.3.

La primera medición (10/11/2026, 25 consultas, 217 s) había dado 24 de 25 intenciones (96 %) y 25 de 25 resultados finales. Su único error fue planificar «¿Cuál es la capital de Francia?» como `GRUPO.parametros`: la validación lo detectó (el texto no tiene ninguna palabra del dominio) y respondió con el mensaje fijo de fuera de alcance sin consultar el grafo. En esa corrida, las consultas sin pedido terminaron en una pregunta («¿Sobre qué pedido?»), nunca en `req_01`, y los modelos inexistentes (Notebook_Z y Notebook_Q) en una aclaración con el catálogo.

Reparo. La medición no es independiente del prompt: cada uno de los 9 ejemplos del prompt maestro tiene su consulta gemela, exacta o muy cercana, en el conjunto de prueba (sección 9.5).

El plan tardó entre 0,7 y 6,1 s en los E2E del 11/11 (una máquina). En otra, la notebook de Ramiro de Coninck (10/11), tardó entre 10 y 50 s; la primera consulta es la más lenta porque Ollama carga el modelo en memoria. Falta medir los tiempos de punta a punta en esa máquina.

7. Casos de uso integrados

La consigna pide al menos tres E2E y cinco casos de uso que **crucen información entre submódulos** (§2, §3.4 y §7). El sistema los resuelve con un plan del router que pide intenciones de más de un submódulo; el orquestador agrega siempre `SM1.interpretacion_pedido` cuando hay un pedido, y eso **no cuenta** como cruce.

Caso	Consulta	Submódulos del plan	Qué muestra
E2E-1	«Para req_01, ¿qué notebook me recomendás? ¿Cómo andan de RAM y de gráfica?»	SM4 + SM2 (+ SM3 según el plan)	Ranking A, H, B; X fuera de presupuesto; M condicionada; E y C descartadas; G indeterminada
E2E-2	«¿Por qué la M quedó condicionada y la G indeterminada? ¿Qué riesgo de RAM tiene la G?»	SM4 + SM2	Los rangos se conservan: M [0,504 ; 0,826] y riesgo de G [49,58 ; 81,46]
E2E-5	«La E tiene casi el mismo puntaje que la B y la H, ¿por qué no la recomiendan?»	SM4 + SM2	La excluyente (REC-R3) gana al puntaje (≈ 0,81)
E2E-9	«¿Qué reglas se activaron para la A, de dónde sale cada dato y qué riesgo de RAM tiene?»	GRUPO + SM2	Trazabilidad (§3.7): 18 reglas de los 4 submódulos, 14 con origen y 4 «no registrada»
E2E-10	«¿Cuáles son las mejores en movilidad y cómo queda el costo-beneficio?»	SM3 + SM4	Orden por movilidad con los datos faltantes al final; índice costo-beneficio de REC-R9

Además hay dos **controles**, que no cuentan como casos integrales: E2E-3 (pedido con perfil indeterminado: el sistema no asume un perfil y pide responder la pregunta pendiente) y E2E-8 (fuera de alcance: mensaje fijo y ninguna consulta al grafo).

Cómo se decide que un E2E aprueba. No por lo que dice el LLM, sino por lo que devuelve el grafo: cada E2E tiene esperados (puestos, estados, rangos, alertas, conteos de reglas) que se comparan con la evidencia de la traza. La explicación en lenguaje natural la escribe el LLM y la revisa un verificador contra la misma evidencia.

Detalle: `docs/pruebas/casos_de_uso.md` (los cinco campos de §3.4 por caso) y un `E2E-n.md` por corrida con la entrada, el plan JSON, las consultas Cypher con sus valores, los datos enviados al LLM, los intentos del verificador y el esperado contra el obtenido.

Límites que el informe tiene que decir. (1) El router con `qwen2.5:3b` no siempre pide el mismo conjunto de submódulos (E2E-1 a veces omite SM3); el aprobado exige al menos dos submódulos distintos, no un conjunto exacto. (2) Para E2E-2 y E2E-5 se agregó a la consulta una segunda pregunta (riesgo de RAM, virtualización) para que crucen submódulos. (3) Los tiempos se midieron en una sola máquina.

8. Pruebas y resultados

Todo se corrió el 11/11/2026 sobre Neo4j 5.26 (instancia local con el grafo cargado desde `cypher/`) y Ollama con `qwen2.5:3b`, temperatura 0.

8.1 Resumen

Prueba	Qué cubre	Resultado
`python -m unittest`	Submódulos, grafo, router, orquestador, explicador, API	171 tests, 2 salteados (los que piden Ollama)
`python -m scripts.verificar_grafo`	Casos de prueba CP1 a CP11 del grafo	11 de 11
`python -m scripts.contar_grafo`	El grafo dividido (`cypher/`) contra el canónico	343 nodos y 683 relaciones, coinciden
`python -m scripts.correr_e2e`	Los 7 E2E con el orquestador real	7 de 7 aprueban (5 integrales y 2 controles)
`PG3_TEST_LLM=1 python -m unittest tests.test_router`	Acierto del router sobre 31 consultas	Intenciones 28 de 31 (90 %); resultado final 30 de 31 (97 %)

Las pruebas funcionales por tipo (recuperación, consulta, regla, difuso y respuesta, consigna §3.5) están en `docs/pruebas/funcionales.md`.

8.2 E2E

Detalle en `docs/pruebas/resultados_e2e.md`. Cada E2E aprueba por lo que dice el grafo, no por el texto del LLM.

E2E	Tipo	Origen de la respuesta	Tiempo (s)
E2E-1	integral	llm	7,1
E2E-2	integral	llm	5,1
E2E-3	control	validación	1,3
E2E-5	integral	llm	4,5
E2E-8	control	fuera de alcance	0,7
E2E-9	integral	llm	4,3
E2E-10	integral	llm	4,3

Los tiempos son de **una** máquina (el plan del router es lo que más tarda, entre 2 y 4 s). Falta medirlos en otra.

8.3 Router

Intenciones acertadas: 28 de 31. Las tres que fallan:

- **R13** (responder una pregunta pendiente): es una intención de nivel 2, que el router rechaza a propósito; el modelo elige `interpretacion_pedido`.
- **R26** y **R30**: el plan omite una intención de las esperadas (SM3.buscar y SM4.explicar_notebook). El E2E igual aprueba, porque exige al menos dos submódulos distintos y no un conjunto exacto.

Una mejora del 11/11: el router descarta los filtros de SM3 que el texto no pide (el modelo copiaba los del ejemplo del prompt: `peso_max` y `bateria_min`).

8.4 Explicador y verificador

- 0 de 5 respuestas con explicación salieron de la plantilla fija en la corrida de los E2E: las redactó el LLM y las aprobó el verificador a la primera.
- El verificador **corta lo inventado** (cifras, modelos, reglas, alertas de otra notebook, rangos reducidos

a un número, perfil asumido, recomendar lo descartado), pero **no garantiza que la redacción sea buena**: con `qwen2.5:3b` hay frases torpes y alguna afirmación suelta sin respaldo numérico (por ejemplo, «GPU integrada moderna» en E2E-1). Esa es la limitación principal del modelo chico.

- Pasar a `qwen2.5:7b` mejora la redacción, pero a un integrante le colgaba la PC.

8.5 Lo que no se verificó

- Los tiempos en más de una máquina.
- Un E2E por nivel 2 (pedido nuevo con inferencia): PG3-21 y PG3-22 a 27 no están hechos.

9. Evaluación del sistema

Esta sección analiza con espíritu crítico lo que el asistente hace bien, lo que no y lo que no probamos (consigna §3.6). Todas las cifras salen de las corridas del 11/11/2026 sobre Neo4j 5.26 y `qwen2.5:3b`, registradas en `docs/pruebas/` y resumidas en la sección 8. Donde algo no se midió, se dice.

9.1 Fortalezas

Fortaleza	Evidencia
Lo que se afirma sale del grafo, no del LLM. Cada E2E aprueba por lo que devuelve Neo4j (puestos, estados, rangos, alertas), no por el texto	7 de 7 E2E aprueban: 5 integrales y 2 controles
Cruza submódulos de verdad. Los casos integrales piden al menos dos submódulos distintos sin contar SM1, que el orquestador agrega siempre	E2E-1, 2, 5, 9 y 10: SM2 + SM4, SM2 + SM4, SM2 + SM4, GRUPO + SM2 y SM3 + SM4
La incertidumbre no se tapa. Un dato faltante se informa como rango o como «falta verificar»	M: grado [0,504 ; 0,826] y `Recomendada condicionada por falta de informacion`. G: riesgo de RAM [49,58 ; 81,46] e `Indeterminada: completar informacion`. `req_02`: perfil indeterminado, sin asumir oficina (E2E-3)
Un requisito excluyente no se compensa con puntaje.	E queda `Descartada (excluyente incumplido)` aunque puntúa ≈ 0,81, casi igual que B (0,813) y H (0,811)
El router se equivoca sin hacer daño. La validación en Python descarta intenciones, modelos y pedidos inválidos antes de tocar el grafo, y nunca asume `req_01`	Una consulta fuera de dominio termina en un mensaje fijo, sin consultar el grafo (E2E-8: 0,69 s). Los modelos inexistentes terminan en una aclaración
Cada respuesta es auditable. La traza guarda plan, Cypher con parámetros, filas, evidencia, intentos y tiempos	Sección 10: reconstrucción completa del E2E-9
Es reproducible. El grafo dividido por dueño equivale al canónico, y la regresión corre sola	343 nodos y 683 relaciones iguales (57 de 57 filas); CP1 a CP11 en 11 de 11; 171 tests (2 salteados, los que piden Ollama)
Corre local y sin costo de API, en una notebook común	`qwen2.5:3b` ocupa unos 4 GB de RAM

9.2 Limitaciones

Los parámetros no están validados por el técnico. Las 47 etiquetas difusas (centros y sigmas), los parámetros del grafo y los mínimos de RAM por perfil están marcados `'sin validar'`. Las consultas al técnico que aparecen en la traza del E2E-9 figuran como `pendiente`. Por eso toda respuesta lleva la advertencia de que los grados y riesgos son una referencia, no una certeza. Los resultados no se pueden presentar como una decisión de compra.

El asistente lee, no infiere en vivo. Las reglas y el difuso corren una vez, al cargar el grafo, y el asistente consulta lo que quedó calculado para tres pedidos (`req_01`, `req_02` y `req_03`) y nueve notebooks. Un comprador nuevo no se puede registrar: `SM1.registrar_pedido` y `SM1.aclarar_pedido` son de nivel 2 y el router las rechaza. Es la limitación más importante frente al objetivo de «realizar inferencias» de la consigna (§1), y el motivo por el que el sistema es una demostración y no una herramienta de uso diario.

El modelo chico limita la redacción. `qwen2.5:3b` planifica bien y redacta regular. Con poco contexto, la evidencia se compacta antes de dársela (listas cortadas a 12 filas): en el E2E-9 el LLM vio 12 de las 18 reglas. Probar `qwen2.5:7b` mejora la redacción, pero a un integrante se le colgó la computadora, y no se adoptó.

Catálogo y pedidos chicos. Son 9 modelos con precios al 01/10/2026 y 3 pedidos. Los precios envejecen y no hay consulta en tiempo real. Una pregunta abierta desde PG1 sigue sin respuesta: si los pesos y umbrales dan rankings razonables sobre un catálogo más grande (el catálogo real prevé 30 modelos).

Sesión mínima. El asistente recuerda solo el pedido de la sesión, no la conversación: cada pregunta se

resuelve sola. El plan admite hasta tres consultas.

Tiempos de punta a punta medidos en una sola máquina. En la máquina de los E2E, las respuestas que redacta el LLM tardaron entre 4,3 y 7,1 s; las de control, 0,7 y 1,3 s. Entre el 93 % y el 99 % de ese tiempo lo consumen las dos llamadas al LLM, y las consultas a Neo4j tardaron entre 0,04 y 0,3 s. El primer E2E es el más lento (6,1 s de plan), algo compatible con la carga del modelo, pero no lo medimos por separado. En la notebook de Ramiro de Coninck (10/11), el plan **solo** tardó entre 10 y 50 s, así que en otro equipo una consulta completa puede llevar de varios segundos a más de un minuto, como avisa el manual. No sabemos qué explica la diferencia entre las dos máquinas, y no hay una medición de punta a punta en la segunda.

9.3 Situaciones problemáticas observadas

Son casos reales de las corridas. En todos, **el E2E aprobó**, porque aprueba por la evidencia del grafo: lo que falla es el texto que lo acompaña.

1. **Un número de otro concepto (E2E-2).** Se preguntó por el riesgo de RAM de la G. El grafo da el rango [49,58 ; 81,46]. La respuesta dijo «un riesgo de RAM del 52.9%». Ese 0,529 es la adecuación global de la G, otra magnitud. El verificador lo dejó pasar porque la cifra existe en la evidencia de esa notebook, y además perdió el rango. Es el error más serio que encontramos: un dato verdadero con el nombre equivocado.
2. **Una respuesta más pobre que su traza (E2E-9).** Se preguntó qué reglas se activaron y de dónde sale cada dato. La traza tiene 18 reglas, 14 con origen y 4 «no registrada». La respuesta nombra 3 reglas (FZ-R2, FZ-R4 y FZ-R5, las de SM2), dice que «los datos vienen de la evaluación del grupo (PI3)» y no menciona la fuente del precio ni de la autonomía ni las reglas sin origen.
3. **Una respuesta que no contesta lo que se pidió (E2E-10).** Se pidieron las mejores en movilidad y el costo-beneficio. La respuesta dice «Vos te conviene la Notebook_A, está recomendada (Alta) en movilidad», mezcla el estado de recomendación con la movilidad, no da el orden y no aclara que la B tiene un índice de costo-beneficio mayor (1,161 contra 0,907 de la A).
4. **Redacción vaga o torpe (E2E-1 y E2E-5).** Para la pregunta «¿cómo andan de RAM?», la respuesta dice «la RAM es mixta», que es el tipo de memoria y no el riesgo. En el E2E-5 aparece «cumpliendo con la alerta excluyente». Nada de eso se contradice con el grafo, pero un asesor lo leería como confuso.
5. **El plan del router a veces omite una intención.** En el E2E-1 falta `SM3.buscar` y en el E2E-9, `SM4.explicar_notebook`. Por eso el criterio de aprobado exige al menos dos submódulos distintos y no un conjunto exacto.
6. **El desempate puede contradecir el costo-beneficio.** La H (grado 0,626, ampliabilidad alta) queda segunda y la B (0,632, ampliabilidad baja) tercera, porque sus grados caen en la misma franja de 0,05 (`P_MARGEN_EMPATE`) y desempata la ampliabilidad. Eso sale de una entrevista y de una decisión del grupo, pero la B rinde más por dólar (1,161 contra 0,811). Cambiar el margen cambia el orden: es un ejemplo de por qué importa validar los parámetros.

9.4 Posibles errores y qué los cubre

Error posible	Qué lo detecta o lo limita	Qué no cubre
El router elige una intención que no existe, un modelo inexistente o un pedido inventado	`validar()` en Python: intención del conjunto cerrado, modelo del catálogo, fragmento literal, pedido existente	Una intención válida pero equivocada, o que falta (casos R26 y R30 del router)
La respuesta cita una cifra, un modelo o una regla inventados	Verificador (controles a a h): regenera hasta 2 veces y después usa la plantilla fija	Una cifra real atribuida a otro concepto (lo encontramos en el E2E-2). Siguen abiertos desde la revisión de código de PG3-13 otros dos: una afirmación que contradice un dato sin nombrar notebook ni estado (por ejemplo, «los parámetros están validados») y el parseo de números con separador de miles (`1.200` se lee como 1,2)
Un rango se reduce a un número	Control (f) del verificador	Lo que no se dice como rango pero lo parece, como el caso del 52,9 %

Un parámetro mal calibrado cambia un estado	La advertencia de `sin validar` en cada respuesta	Nada hasta que el técnico los valide
Un precio o una ficha desactualizados	La fuente y la fecha del precio salen en la traza	No hay actualización automática
Ollama o Neo4j no responden	El sistema devuelve `origen_respuesta = "error"` con un mensaje fijo	—
Cargar el grafo en una base compartida	`python -m scripts.cargar_grafo` **borra la base**: se usa solo en una instancia propia	No hay confirmación antes de borrar
Subir una credencial en el ZIP	`armar_zip` se niega si encuentra una contraseña o una URI de Aura	El control es por patrón, no reemplaza una lectura final

9.5 Cómo leer las cifras del router

El router acertó las intenciones en 28 de 31 consultas (90 %) y el resultado final en 30 de 31 (97 %). Hay que leerlo con un reparo: **la medición no es independiente del prompt maestro**. Seis de las 31 consultas (R01, R02, R05, R14, R17 y R18) coinciden, salvo las tildes, con ejemplos que están en el prompt, y otras tres (R10, R11 y R29) son variantes muy cercanas. Las tres que fallan (R13, R26 y R30) están todas entre las otras 22, de modo que sobre esas el acierto de intenciones es 19 de 22 (86 %), una cifra más honesta de lo que generaliza el modelo. Es un cálculo nuestro a partir de la lista de fallas de la sección 8.3. Tampoco existe todavía un conjunto de consultas escritas por alguien que no vio el prompt.

9.6 Oportunidades de mejora

Mejora	Qué resuelve	Referencia
Validar los parámetros con el técnico, con una tabla de escenarios de RAM y las consultas `CE-*`	Pasa los parámetros de `sin validar` a validados y permite sacar la advertencia	PG1, `ConsultaExperto`
Registrar pedidos nuevos con inferencia en vivo (`registrar_pedido` y `aclarar_pedido`, con el Cypher parametrizado por pedido)	Que el asistente razone para un comprador real y no solo lea	D-08, PG3-21
Completar las reglas del nivel 2: canal de memoria y otras reglas de SM2 (REN-R2, R11, R13, R14), intensidad a gama (GM-R1, R2, R14 a R16), requisitos explícitos y dos escenarios de SM1, monitor externo (REC-R16)	Cerrar lo que PG1 y PG2 describieron y el grafo todavía no tiene	D-04, D-09, D-10; PG3-22 a 24 y 26
Cargar `SALIDA_PI2` de los 7 perfiles, `req_04` y `req_05`, y el caso de dos compradores (E2E-4)	Más pedidos y más cobertura de las pruebas	D-03, D-11; PG3-25
Verificador por concepto: atar cada cifra al campo del que sale, cortar afirmaciones que contradicen un dato y arreglar el parseo de miles	El error del E2E-2 y sus parientes	Abiertos del review de PG3-13
Probar un modelo más grande (`qwen2.5:7b` o `llama3.1:8b`, ya permitidos)	Mejor redacción, a costa de hardware	`docs/RESTRICCIONES.md` §4
Medir el router con consultas independientes y repetir los tiempos en otras máquinas	Cifras que generalicen	Sección 9.5
Guardar la fuente en cada regla y agregar nodos de fuente con referencia concreta	Cerrar los 4 «no registrada» y llegar de la regla a una cita	Sección 10, D-13, PG3-27
Catálogo de 30 modelos con precios actualizados	Sacar al sistema de la demostración	PG1

9.7 Lo que no está hecho o no se verificó

Al 11/11/2026, cuando se redactó este informe:

Qué	Estado
Pedido nuevo con inferencia en vivo (PG3-21)	No se hizo. El sistema lo declara fuera de su alcance
Nivel 2 (PG3-22 a 27)	No se hizo. Es trabajo futuro, con su diseño ya documentado en los contextos
Tiempos de punta a punta en otra máquina	No se midieron (solo el plan, de 10 a 50 s, en la notebook de Ramiro de Coninck)
Video demostrativo (PG3-19)	Pendiente, lo hace una persona
Infografía (PG3-18)	Se cubre con el resumen ejecutivo de una página (`docs/presentacion/resumen_ejecutivo.pdf`)
Regeneración del verificador con el modelo real	Probada con un LLM falso (`tests/test_explicador.py`). En las 5 respuestas reales de los E2E el verificador aprobó a la primera, así que ninguna usó la regeneración ni la plantilla
Reproducción desde un ZIP limpio siguiendo el manual	No se hizo
Revisión de código (`/code-review`) sobre PG3-11	No se corrió

10. Trazabilidad del conocimiento

La consigna (§3.7) pide poder reconstruir, para una conclusión del sistema, cinco cosas: la fuente del conocimiento, la regla, la evidencia, la consulta y la explicación. Esta sección explica dónde queda cada una y lo demuestra con **un ejemplo real reconstruido de punta a punta**, el E2E-9, con sus límites a la vista.

10.1 Dónde queda cada eslabón

Eslabón	Dónde se encuentra
Fuente	En el grafo: `Parametro.origen` (incluidos los `centros_y_sigmas_<variable>` que respaldan cada regla difusa), `EtiquetaDifusa.fuente`, `Juego.fuente`, `Notebook.fuente_precio` y `Bateria.fuente`, más las consultas pendientes al técnico (`ConsultaExperto`). El submódulo GRUPO las expone con `GRUPO.origen_reglas` y `GRUPO.origen_notebook`. Lo que el grafo no tiene sale como «no registrada», sin inventar nada
Regla	Cada regla es un nodo `Regla`, y cada activación, una relación `[:ACTIVA {fuerza}]`. La consulta `GRUPO.L3` las devuelve para una notebook en un pedido
Evidencia	La evidencia común que recibe el explicador, guardada completa en la traza
Consulta	El Cypher fijo con sus parámetros, las filas que devolvió y su duración, en la traza
Explicación	La respuesta final y cada intento del verificador con los problemas que encontró, en la traza

La traza es una línea JSON por consulta del usuario, en `trazas/AAAA-MM-DD.jsonl` (esa carpeta no se versiona). `docs/pruebas/E2E-n.md` presenta la de cada E2E paso a paso, y `datos/traza_ejemplo.jsonl` guarda una en formato crudo (la del E2E-1). La del E2E-9, que reconstruimos a continuación, se lee completa en `docs/pruebas/E2E-9.md`.

10.2 Ejemplo reconstruido: E2E-9

Consulta del asesor: «En req_01, ¿qué reglas se activaron para la A, de dónde sale cada dato y qué riesgo de RAM tiene?» (corrida del 11/11/2026, 4,3 s, detalle en `docs/pruebas/E2E-9.md`).

Paso 1 · El plan del router

El LLM devolvió un plan con dos intenciones, y el orquestador agregó `SM1.interpretacion_pedido`:

```
{"requerimiento_id": "req_01",
 "consultas": [
   {"intencion": "GRUPO.traza_reglas", "parametros": {"modelo": "Notebook_A"}},
   {"intencion": "SM2.riesgo_ram", "parametros": {"modelos": ["Notebook_A"]}}]}
```

La validación anotó una observación: descartó un fragmento por no ser literal (el texto dice «la A» y el plan escribió «Notebook_A»). El plan siguió siendo válido. Es el tipo de detalle que la traza deja ver y que una respuesta sola esconde.

Paso 2 · Las consultas ejecutadas

Seis consultas Cypher fijas, todas de solo lectura: `SM1.L1` (el pedido), `GRUPO.existe_evaluacion`, `GRUPO.L3` (las reglas activadas, 20 filas), `GRUPO.origen_reglas` (19 filas), `GRUPO.origen_notebook` y `SM2.C2` (el riesgo de RAM, 1 fila, 12 ms). La L3, abreviada y con saltos de línea para el informe, es así:

```
MATCH (c:Requerimiento {id:$requerimiento_id})-[:EVALUA]->(ev:Evaluacion)
      -[:SOBRE]->(n:Notebook {modelo:$modelo})
OPTIONAL MATCH (c)-[:INCLUYE]->(a:Actividad)
WITH c, ev, n, collect(a) AS acts
UNWIND [c, ev, n] + acts AS origen
OPTIONAL MATCH (origen)-[:TIENE_LECTURA*0..1]->(x)-[act:ACTIVA]->(q:Regla)
RETURN q.submodulo AS submodulo, q.id AS regla, q.tipo AS tipo, act.fuerza AS fuerza,
       q.estado_validacion AS validacion  // y el antecedente y el consecuente
```

Los parámetros son `{"requerimiento_id": "req_01", "modelo": "Notebook_A"}`. Las 20 filas que devuelve se agrupan en **18 reglas distintas de los cuatro submódulos**: 5 de SM1, 3 de SM2, 3 de SM3 y 7 de SM4. El texto exacto de cada consulta y sus primeras filas están en `docs/pruebas/E2E-9.md`.

Paso 3 · La regla

Tomamos una de las tres reglas de SM2 que activó la A, la **FZ-R4** (difusa, Mamdani, `sin validar`):

> `MIN(S.justa, MAX(A.media, A.mucha))` → `riesgo_obsolescencia = bajo`

Es decir: si la RAM es justa para el perfil y la ampliabilidad es media o mucha, el riesgo es bajo. Para la A, con 16 GB de RAM mixta y ampliable, ρ = 1,0 (la RAM iguala al mínimo del perfil de virtualización) y un índice de ampliabilidad de 0,75, la FZ-R4 se activó con **fuerza 0,458**. La FZ-R2 y la FZ-R5 también, pero casi sin peso (0,044 cada una). La agregación y el centroide dan un riesgo de **23,02** sobre 100, etiqueta `bajo`.

Paso 4 · La fuente

Desde la regla se llega a su origen por el parámetro que la respalda (consulta `GRUPO.origen_reglas`):

Eslabón de la cadena	Valor en el grafo
Regla	`FZ-R4`, consecuente `riesgo_obsolescencia`
Parámetro	`centros_y_sigmas_riesgo_obsolescencia`
Origen	«Decision de diseno del grupo (PI3)»
Estado de validación	`sin validar`
Consulta al técnico	`CE-riesgo_obsolescencia`, estado `pendiente`

Los datos de la notebook, en cambio, sí tienen fuente externa (`GRUPO.origen_notebook`): el precio sale de «Mercado Libre Argentina y tiendas oficiales, 01/10/2026» y la autonomía, de «Notebookcheck / RTINGS».

Paso 5 · La evidencia

El explicador no vio el grafo: recibió una evidencia común de **5434 caracteres** con el pedido (perfil dominante VIRT, grado 0,899; GAM 0,313), la traza de reglas con su origen, el riesgo de RAM de la A (23,02, `bajo`, con sus tres reglas) y la advertencia de que los parámetros están sin validar. Por el poco contexto del modelo, la lista de reglas se compactó: el LLM vio 12 de las 18 y la marca `_omitidas: 6`. La evidencia incluye además `reglas_sin_origen`: GM-R3, INT-R11, REC-R4 y REC-R9.

Paso 6 · La explicación

El verificador aprobó la respuesta a la primera (origen `llm`):

> Para la A, se activaron las reglas FZ-R2, FZ-R4 y FZ-R5. Los datos vienen de la evaluación del grupo (PI3). La RAM tiene un riesgo de 23.02, que es bajo. Los parametros difusos y los umbrales del sistema estan sin validar por el tecnico: los grados y riesgos son una referencia, no una certeza.

Las cifras (23,02 y `bajo`) y las tres reglas citadas coinciden con la consulta `SM2.C2`. Pero la respuesta es **más pobre que su traza**: nombra 3 de las 18 reglas, resume el origen en una frase y no dice nada de las cuatro reglas sin origen ni de la fuente del precio y de la autonomía. Lo auditable es la traza, no el texto.

Veredicto del E2E

El E2E-9 aprueba por lo que dice el grafo: la traza de la A tiene reglas de los 4 submódulos, 18 reglas, 14 con origen y 4 «no registrada» (GM-R3, INT-R11, REC-R4 y REC-R9); la notebook trae la fuente del precio y de la autonomía; y el riesgo de RAM es 23,02 `bajo`, tal cual lo da el grafo.

10.3 Cómo auditar una conclusión

Para una respuesta cualquiera, el camino es el inverso del ejemplo:

1. Se abre la traza de la consulta y se lee el **plan**: qué intenciones se pidieron y con qué parámetros.
2. Se leen las **consultas**: el Cypher, los parámetros y las filas devueltas. Cada cifra de la respuesta tiene que estar en esas filas.
3. Se mira la **evidencia** que recibió el explicador y los **intentos** del verificador: qué problemas encontró y cuántas veces regeneró.
4. Desde cada regla citada se sube a su **origen** con `GRUPO.origen_reglas`, y desde la notebook, a sus fuentes con `GRUPO.origen_notebook`.

Se puede repetir con la consola (`python -m asistente.cli --traza "..."`) o corriendo `python -m`

`scripts.correr_e2e`, que reescribe `docs/pruebas/`.

10.4 Límites de la trazabilidad

Lo que el sistema **no** permite reconstruir hoy, dicho sin vueltas:

- **14 de las 18 reglas de la traza de la A tienen origen y 4 salen «no registrada»** (GM-R3, INT-R11, REC-R4 y REC-R9). El grafo canónico no guarda `fuente` en los nodos `Regla` y no tiene nodos de fuente.
- **El origen que se muestra es el de los parámetros, no el de la regla.** Se llega a él por el nombre del parámetro (`centros_y_sigmas_<variable>`) o porque el origen del parámetro menciona la regla. Es una cadena indirecta.
- **Ese origen es una decisión del grupo, no una cita.** Las seis entradas de origen de este ejemplo son «Decision de diseno del grupo (PI3)» o «Decision del grupo (REC-R5)», todas `sin validar`. El sistema puede decir que un parámetro lo decidió el grupo y que su validación está pendiente; **no puede** llevar de la regla a una ficha, a un manual o a la entrevista con el técnico. Los datos de catálogo (precio, autonomía) sí tienen fuente externa.
- **El texto de la respuesta no es evidencia.** Puede resumir mal o callar cosas, como en este ejemplo. La prueba es la traza.
- **Se reconstruyó un ejemplo, no todos.** La traza se guarda para cada consulta, pero la reconstrucción manual la hicimos para este caso. El E2E-9 se corrió una vez; el texto puede variar entre corridas, la evidencia no.

11. Reflexiones finales

11.1 Reflexión sobre la metodología

Formulación del problema. Definir desde PG0 que el usuario es el asesor y el beneficiario el comprador fijó el tono y el destinatario de toda explicación. También fijó el alcance: hasta tres notebooks, con alertas y una explicación trazable. Esa decisión llegó hasta PG3: el explicador redacta para el asesor, en voseo y con un máximo de 180 palabras (D-07).

Análisis de tareas expertas. Descomponer la tarea del técnico en interpretar el pedido, consolidar requisitos, evaluar rendimiento, gráfica y movilidad, filtrar, ordenar y explicar dio los cuatro submódulos y su orden. La regla de PG1 «primero se filtran los excluyentes, después se trata la información incompleta» es la que hoy descarta a Notebook_E pese a su adecuación de 0,81 (CP2, E2E-5).

Adquisición de conocimiento. Las fichas oficiales, los requisitos de software, Notebookcheck y la entrevista al técnico dieron los mínimos por perfil y el tratamiento de datos faltantes. La lección más fuerte fue separar «falta información» de «no cumple»: Notebook_G no informa virtualización ni ampliabilidad, y por eso queda `Indeterminada: completar informacion`, no descartada (CP4).

Modelado conceptual. La red semántica y los frames de PG1 se tradujeron casi uno a uno a labels y relaciones de Neo4j. La corrección de PG2, que guarda todo lo que depende del pedido en una `(:Evaluacion)`, permite que dos compradores evalúen la misma Notebook_K sin pisarse: para `req_01` queda `Descartada` y para `req_03`, `Recomendada (Media)` con grado 0,469 (CP11).

Lógica difusa. Las 23 reglas Mamdani manejan la vaguedad del dominio: frecuencias («a veces»), «liviana», «RAM justa». El riesgo de RAM de FZ-R1..R5 distingue a Notebook_K (49,55, ranura libre) de Notebook_G, que sin dato de ampliabilidad se informa como rango [49,58 ; 81,46] en lugar de un número falso. Su límite también es metodológico: los 77 parámetros siguen `'sin validar'` por el técnico, y el sistema lo advierte en cada respuesta.

Implementación en grafos. Tener cada regla como nodo y cada activación como `[:ACTIVA {fuerza}]` vuelve recorrible la inferencia. La traza de Notebook_A en `req_01` (consulta L3) devuelve 20 filas que son 18 reglas distintas de los cuatro submódulos, con su fuerza y, en 14 de ellas, su origen (sección 10). Dividir el grafo en un archivo por dueño, verificado con la misma cantidad de nodos y relaciones por tipo (57 de 57) y la regresión CP1–CP11 en 11 de 11, permitió que cuatro personas trabajaran en paralelo sin pisarse.

Integración con el LLM. Siguiendo el ejemplo de PG3 de la cátedra, el LLM es solo la interfaz: un router con prompt maestro arma un plan JSON (LangChain + Ollama, `qwen2.5:3b` a temperatura 0, salida estructurada) y Python lo valida antes de tocar el grafo. Las consultas Cypher son fijas y las elige el código. La validación en Python fue necesaria: sobre 31 consultas de prueba el modelo acertó las intenciones en 28 (90 %) y el resultado final, ya validado, fue correcto en 30 (97 %). En la primera medición, la consulta que el modelo erró («¿Cuál es la capital de Francia?», planificada como consulta de parámetros) la corrigió la validación, que la devolvió como fuera de alcance sin tocar el grafo (sección 6). El otro extremo del sándwich, el explicador, mostró el límite del enfoque: el verificador corta lo inventado pero no garantiza una buena redacción (sección 9).

11.2 Conclusiones del grupo

Lo que logramos. Construimos un asistente que no es una suma de scripts: un prompt maestro que enruta cada consulta, una única entrada (API y consola), cuatro submódulos que cumplen el mismo contrato y comparten un grafo en Neo4j, y cinco casos integrales que cruzan al menos dos submódulos distintos. La idea central de la materia se sostiene en el código: el LLM interpreta y explica, y el conocimiento verificable decide.

Contra los objetivos de la consigna (§1).

Objetivo	Estado	Qué lo respalda
Interpretar consultas	Cumplido	Router más validación: 28 de 31 intenciones (90 %) y 30 de 31 resultados finales (97 %); las consultas fuera de dominio y los pedidos inexistentes se controlan sin tocar el grafo

Recuperar conocimiento	Cumplido	11 intenciones con Cypher fijo sobre un grafo de 343 nodos y 683 relaciones; CP1 a CP11 en 11 de 11
Realizar inferencias	**Parcial**	Las reglas crisp y las difusas Mamdani corren al cargar el grafo y el asistente consulta lo que dejaron calculado. No infiere en vivo para un pedido nuevo (PG3-21 no se hizo)
Gestionar la incertidumbre	Cumplido	Rangos, `desconocida` e indeterminado llegan intactos a la respuesta; los parámetros sin validar se advierten en cada una
Explicar de manera comprensible	Cumplido, con reparos	Las 5 explicaciones pasaron el verificador, pero con redacción pobre y un error de atribución (E2E-2). La traza, en cambio, es completa

Lo que aprendimos.

1. **Aprobar por la evidencia y no por el texto fue una buena decisión, pero insuficiente.** Probar contra lo que devuelve el grafo nos dio pruebas estables aunque el LLM redacte distinto cada vez. A la vez, un E2E aprobado convivió con una explicación equivocada. El verificador controla que una cifra exista, no que tenga el nombre correcto.
2. **Lo que importa se verifica en código, no en el prompt.** Un modelo de 3 mil millones de parámetros tiende a ignorar las instrucciones secundarias; lo que lo contuvo fue el plan validado, las consultas fijas y el verificador.
3. **«Falta información» no es «no cumple».** La lección de dominio más fuerte: la G queda indeterminada y la M, condicionada, en lugar de descartadas o completadas con un dato inventado. Mostrar un rango en vez de un número falso es más incómodo y más honesto.
4. **Un contrato fijo permitió trabajar en paralelo.** La misma interfaz, fixtures con filas reales y un único punto de entrada dejaron que cuatro personas avanzaran a la vez. Antes de integrar hubo que resolver 13 decisiones (D-01 a D-13) por inconsistencias entre los contextos de cada submódulo: reconciliar eso fue trabajo, no un trámite.
5. **Hay que medir con consultas que el modelo no vio.** Nuestra primera medición del router (25 consultas, 96 %) fue más optimista que la final (31 consultas, 90 %), y después advertimos que cada ejemplo del prompt tiene su gemela, exacta o muy cercana, en el conjunto de prueba. Convendría haber separado desde el principio las consultas de ejemplo de las de evaluación.
6. **Una sola fuente para las cifras.** Al armar este informe encontramos cifras desfasadas entre documentos (la cantidad de consultas del router, las filas contra las reglas de la traza). Las unificamos contra las corridas de `docs/pruebas/`, que son la referencia.
7. **El modelo chico es una decisión de compromiso.** `qwen2.5:3b` corre en la notebook de cada integrante, sin costo, y alcanza para enrutar. No alcanza para redactar con soltura. Un modelo más grande mejoraría el texto, pero a un integrante se le colgó la computadora.

Lo que no logramos, y lo que sigue. Dejamos sin hacer el pedido nuevo con inferencia en vivo y todo el nivel 2 (PG3-21 a 27), no medimos los tiempos de punta a punta en otra máquina y los parámetros siguen sin validar por el técnico. Esto último es lo que más pesa: hasta que no se validen, el sistema es una demostración funcional de la arquitectura y no una herramienta que alguien deba usar para decidir una compra. Si siguiéramos, el orden sería: validar los parámetros con el técnico, habilitar el pedido nuevo con inferencia en vivo, reforzar el verificador para que ate cada cifra a su concepto y recién después ampliar el catálogo y las reglas. El detalle está en la sección 9.

12. Referencias y fuentes consultadas

Esta lista junta lo que figura en el repositorio y en los informes previos del grupo. No agregamos ninguna referencia que no hayamos usado.

Material de la cátedra

- Cátedra de Inteligencia Artificial, UTN FRM (2026). *Actividad PG3 – Unidad 4 (Grupal): orquestación final, pruebas del prototipo y armado de carpeta* (consigna). Transcripta en `docs/consigna/ConsignaPG3.md`.
- Cátedra de Inteligencia Artificial, UTN FRM (2026). *Ejemplo_PG3_Evaluacion_CVs.docx*: ejemplo de entrega del Grupo 1 (evaluación de CV), usado como guía de forma.
- Cátedra de Inteligencia Artificial, UTN FRM (2026). *Mapa de la Materia: Entregas, Herramientas y Metodología (U0 a U4)*, y apuntes de la Unidad 4 (U1a, U1b y U1e). Base de `docs/RESTRICCIONES.md`.
- Entregas de la cátedra A0 a A11 y actividades PI1 a PI4 y PG0 a PG3 del Grupo 2, en particular A10 (prompt engineering) y A11 (Graph RAG con Ollama y Neo4j).

Trabajos previos del grupo

- Grupo 2 (2026). *PG1*: cuaderno de conocimiento, red semántica y macro-proceso (`Grupo_2_PG1_U1.docx`, no incluido en este repositorio).
- Grupo 2 (2026). *PG2*: informe y script `Grupo_2_PG2_U1.cypher`. La versión canónica corregida es `docs/cypher/Grupo_2_PG2_U1_corregido.cypher` (decisión D-01).
- Entregas PI4 de cada submódulo, en `docs/fuentes/`: `Olea_Facundo_PI4_U4.ipynb` (SM1), `Miranda_Mauricio_PI4_U4.py` (SM2), `sm3_asistente.py` y sus prompts (SM3) y `Luconi_Martina_PI4_U4.py` (SM4).
- Contextos para PG3 en `docs/contexto/`: `Contexto_PG1_PG2_Grupo_para_PG3.md` y `Contexto_SM1_Olea_para_PG3.md`, `Contexto_SM2_Miranda_para_PG3.md`, `Contexto_SM3_deConinck_para_PG3.md` y `Contexto_SM4_Luconi_para_PG3.md`.
- Documentos de PG3 del repositorio: `docs/ARQUITECTURA.md`, `docs/RESTRICCIONES.md`, `docs/INCONSISTENCIAS_CONTEXTOS.md` (decisiones D-01 a D-13), `docs/plan/PLAN_PG3.md`, `docs/CORRECCIONES.md` y las notas de `docs/sesiones/`.
- Corridas citadas en este informe, en `docs/pruebas/`: `resultados_e2e.md`, `E2E-n.md` (E2E-1, 2, 3, 5, 8, 9 y 10), `casos_de_uso.md`, `funcionales.md` y `conteos_grafo.md`.

Fuentes de conocimiento del dominio

Son las que el grupo registró en el cuaderno de conocimiento de PG1 para construir los perfiles, los requisitos, los parámetros y el catálogo. Las URL y las fechas de consulta de cada una figuran en los informes de PG1 y PG2, que no están en este repositorio.

- Apuntes y guía de la cátedra.
- Requisitos oficiales de software: Docker, Adobe, DaVinci, PyTorch y Steam.
- Fichas de fabricantes: Intel ARK, AMD, NVIDIA y Lenovo PSREF.
- Notebookcheck y RTINGS (autonomía y mediciones).
- Entrevista a un técnico (mínimos por perfil, margen de empate y tratamiento de datos faltantes). La validación de los parámetros con el técnico sigue pendiente (sección 9.2).
- Casos publicados (r/SuggestALaptop).
- Mercado Libre y tiendas oficiales (precios al 01/10/2026) y cotización del Banco Nación.

Bibliografía técnica

- Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3).
- Mamdani, E. H. y Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1).

- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.
- Neo4j, Inc. *Cypher Manual* y *Neo4j Python Driver Manual*. https://neo4j.com/docs
- Ollama. Documentación de la API y de las salidas estructuradas. https://ollama.com

Software utilizado

Python 3.14 y su biblioteca estándar (`unittest`, `json`, `csv`); Neo4j 5.26 y su driver de Python (`neo4j`); Ollama con el modelo `qwen2.5:3b`; LangChain (`langchain-core` y `langchain-ollama`); FastAPI, Uvicorn y Pydantic; `requests` y `httpx`. Las versiones mínimas están en `requirements.txt`. Todo está dentro de las listas verde y amarilla de `docs/RESTRICCIONES.md`, salvo la excepción registrada en su sección 8 (`langchain-ollama`).

Para armar este informe se usaron además `matplotlib` (las dos figuras), `mistune` y PyMuPDF (el PDF), que no son parte del sistema y no están en `requirements.txt`. Se usan en `docs/informe/figuras.py` y `docs/informe/armar_pdf.py`.

Anexo A · Registro de correcciones (consigna §3.1)

Este registro se redactó el 10/11/2026 y algunos pasajes quedaron desactualizados: dice que SM4, el explicador y el verificador siguen como stub, que los E2E no se ejecutaron y cita la medición del router de ese día (24 de 25). Ya están hechos. Las cifras vigentes (router 28 de 31 y 30 de 31, los 7 E2E) están en las secciones 6 y 8.

> **Anexo del informe final** · Grupo 2 · PG3 · tarea PG3-15 · redactado el 10/11/2026. **Qué documenta.** La revisión y depuración final que pide la consigna §3.1: qué se detectó, qué se corrigió, dónde quedó la corrección y cómo se comprobó. Las fuentes son `INCONSISTENCIAS_CONTEXTOS.md` (S-01 a S-06 y D-01 a D-13), las notas de `docs/sesiones/`, el historial de `git` y los resultados de las corridas que esas notas citan.

Cómo leer este registro

- **Los seis aspectos de §3.1** se numeran **A1** coherencia conceptual, **A2** consistencia terminológica, **A3** funcionamiento de las consultas, **A4** funcionamiento de las reglas, **A5** funcionamiento de los componentes difusos y **A6** funcionamiento de la integración con el LLM.
- **Una fila por corrección, no por decisión.** Cada fila se cuenta una sola vez, bajo su aspecto principal; si toca otro aspecto, el texto lo dice.
- **Estado.**
  - **Corregida:** el cambio está hecho y tiene una verificación citada.
  - **Parcial:** una parte está hecha y lo que falta está dicho en la fila.
  - **Sin verificar:** el cambio está aplicado, pero ninguna prueba lo ejercita.
  - **Abierta:** el problema se detectó y no se corrigió.
- **†** marca un commit o un archivo que, al redactar este registro, está en las ramas `pg3-05-cierre-trazabilidad` o `docs/validacion-plan-pg3` y todavía no se unió a `main`. Al unirse, los hashes pueden cambiar.
- Los estados del grafo se escriben **sin tildes**, como en el grafo (D-06).
- **Alcance de la comprobación.** En esta redacción no se dispuso de Neo4j ni de Ollama. Lo que requiere esos servicios se cita de las notas de sesión y se marca «no reproducido». Lo que sí se corrió se indica en la sección Límites de este registro.

Resumen

Aspecto de §3.1	Correcciones	Corregida	Parcial	Sin verificar	Abierta	Verificado sin cambios
A1 · Coherencia conceptual	11	6	5	0	0	1
A2 · Consistencia terminológica	9	5	3	0	1	0
A3 · Funcionamiento de las consultas	7	5	1	0	1	1
A4 · Funcionamiento de las reglas	4	1	1	2	0	1
A5 · Funcionamiento de los componentes difusos	3	2	1	0	0	1
A6 · Funcionamiento de la integración con el LLM	9	9	0	0	0	1
Total	**43**	**28**	**11**	**2**	**2**	**5**

Los seis aspectos tienen al menos una corrección y, además, una verificación que no requirió cambios (salvo A2, que se comprueba dentro de sus propias filas: C-15 y C-16). Dos advertencias para el informe:

- **Lo decidido no siempre está implementado.** Varias decisiones de INCONSISTENCIAS corrigieron los textos de los contextos y dejaron el cambio de código o de datos para después. Esas filas figuran como **Parcial**, y la sección Decisiones tomadas y no implementadas las junta.
- **La integración con el LLM está verificada hasta el plan.** El router y la validación tienen medición y tests. El explicador y el verificador comunes (PG3-13) y SM4 (PG3-12) están todavía como stub, y sus correcciones se agregarán cuando existan (ver Pendiente de agregar).

Registro de correcciones

C-01 · A1 · README · Corregida

- **Problema detectado:** El README decía que `docs/contexto/` tenía solo los contextos de SM2 y SM4 (S-03).
- **Corrección:** Se actualizó la descripción de la carpeta.
- **Archivo o commit:** `README.md` · 75124b6
- **Cómo se verificó:** Lectura cruzada del README contra el contenido actual de `docs/contexto/` (seis archivos).

C-02 · A1 · Contexto de SM2 · Corregida

- **Problema detectado:** SM2 conservaba como dudas abiertas cuestiones que el contexto PG1/PG2 ya había resuelto: 81,46 frente a 81,1; fuerza 0,863 frente a grado 0,785; autora de SM4; REN-R16 cargada; TSK de SM3 reemplazado por Mamdani; §8 de SM2 reemplazada por la de PG1/PG2 (S-06).
- **Corrección:** Se marcaron como resueltas en el contexto de SM2.
- **Archivo o commit:** `docs/contexto/Contexto_SM2_Miranda_para_PG3.md` · 75124b6
- **Cómo se verificó:** Contraste con PG1/PG2. El valor 81,46 es el máximo del rango de riesgo de la G en CP4, que da OK en la regresión (11/11, 10/11/2026).

C-03 · A1 · Grafo canónico · Parcial

- **Problema detectado:** Había tres scripts de grafo en dos linajes y cada contexto describía uno distinto (D-01). El linaje A guardaba riesgo, grado de GPU y adecuación gráfica en `RAM`/`Notebook`; el B, por `Evaluacion`. Sin una versión única no se podían fijar los valores esperados.
- **Corrección:** Se adoptó `docs/cypher/Grupo_2_PG2_U1_corregido.cypher` (linaje B más REC-R9 con prioridad «precio» y el bloque 13) como canónico, y se reescribió PG1/PG2 §8 y las referencias de SM1 a SM4. **Queda diferido** portar al canónico las piezas de SM1 del linaje A (INT-R2, R3, R4 y R6, Docker y VirtualBox, `orden_exigencia`, INT-R14 con dos lecturas).
- **Archivo o commit:** c950361 (evidencia), 9080475 (decisión)
- **Cómo se verificó:** Comparación de los tres scripts (D-01 de INCONSISTENCIAS). Carga del canónico en Neo4j 5.26: 343 nodos y 683 relaciones, y CP1 a CP11 en 11/11 (no reproducido hoy).

C-04 · A1 · Contextos / grafo · Corregida

- **Problema detectado:** Los contextos hablaban de 61 reglas, 59 parámetros y 9 expresiones de frecuencia; el canónico tiene otras cantidades (D-01).
- **Corrección:** Los contextos se alinearon con el canónico: 57 reglas (23 difusas, 27 crisp y 7 retiradas), 30 parámetros y 14 expresiones.
- **Archivo o commit:** `docs/contexto/Contexto_PG1_PG2_Grupo_para_PG3.md` · 9080475
- **Cómo se verificó:** Conteo en el grafo cargado: `Regla` 57, `ReglaDifusa` 23, `ReglaCrisp` 27, `Parametro` 30, `ExpresionFrecuencia` 14 (`docs/pruebas/conteos_grafo.md`†, 34f3c4d†). Las 7 retiradas se deducen de 57 – 23 – 27; no hay un conteo directo.

C-05 · A1 · Perfiles (SM2, SM3, SM4) · Corregida

- **Problema detectado:** Los pesos y los requisitos por perfil diferían entre el grafo y los contextos (D-02): DEV pedía RAM 8/16 en SM2 y 16/32 en el grafo; DATOS exigía virtualización en SM2 y no en el grafo; DIS pedía gama ≥ 4 en SM3 y 2 en el grafo.
- **Corrección:** Manda la tabla del canónico. Se corrigieron SM2 §3.5, §5.1 y §6.1 y SM3 §9.2. DATOS no exige virtualización (REN-R1 aplica solo a VIRT). Núcleos, carga sostenida y SSD recomendado quedan fuera del alcance.
- **Archivo o commit:** 8a166cd
- **Cómo se verificó:** Revisión cruzada contra el canónico. Las pruebas automáticas ejercitan solo VIRT (`req_01`) y OFI (`req_03`): los demás perfiles no tienen prueba.

C-06 · A1 · SM2 / SM3 / grafo · Parcial

- **Problema detectado:** SM2 describía el cálculo de la adecuación de rendimiento y SM3 el de pantalla, pero en el grafo ambas son dato de entrada (`SALIDA_PI2`). El canónico solo las carga para VIRT (y K en OFI), de modo que un pedido de otro perfil no evalúa ninguna notebook (D-03).

- **Corrección:** Los contextos declaran rendimiento y pantalla como dato de entrada y no calculado. **Queda diferida** la carga de `SALIDA_PI2` de los 7 perfiles.
- **Archivo o commit:** e140e24
- **Cómo se verificó:** Conteo del grafo: `SALIDA_PI2` = 10 relaciones (`docs/pruebas/conteos_grafo.md`†).

C-07 · A1 · Interfaz entre submódulos · Parcial

- **Problema detectado:** Un submódulo esperaba datos que ninguno producía: `entrena_modelos_gpu`, `aviso_sm3` y el JSON de grados de SM2 a SM4 (D-10).
- **Corrección:** Se descartan `entrena_modelos_gpu` (ninguna regla lo consume) y el JSON de grados: la interfaz entre submódulos es el grafo (`Evaluacion`) y la evidencia común. `aviso_sm3` queda resuelto por las alertas de D-04. **Queda diferido** `requisitos_explicitos`.
- **Archivo o commit:** 28276b0
- **Cómo se verificó:** Búsqueda en `asistente/`, `cypher/`, `scripts/`, `tests/` y `prompts/`: no aparece ninguno de los tres nombres.

C-08 · A1 · Regresión CP1 a CP11 · Corregida

- **Problema detectado:** El primer esperado escrito para CP11 era incorrecto. Para `req_01`, K queda `Descartada` a secas, como dice PG1/PG2 §9.4, y no por excluyente. El error estaba en el esperado; el grafo era correcto. La nota de sesión no conserva el texto original del esperado.
- **Corrección:** El esperado pasó a K `Descartada` en `req_01` y `Recomendada (Media)` con grado 0,469 en `req_03`.
- **Archivo o commit:** `scripts/verificar_grafo.py` (CP11) · 1dee294
- **Cómo se verificó:** `python -m scripts.verificar_grafo`: 11/11 OK (nota de PG3-01), repetido sobre una instancia nueva en PG3-05 (34f3c4d†). No reproducido hoy.

C-09 · A1 · Plan de pruebas (PG3-11) · Parcial

- **Problema detectado:** Solo 3 de los 5 casos «integrales» cruzaban submódulos: E2E-3 usa solo SM1 y E2E-8 ninguno. La consigna (§3.4 y §7) pide al menos 5 casos y 3 E2E que involucren varios submódulos.
- **Corrección:** Se agregaron E2E-9 (traza de reglas, riesgo de RAM y explicación de la A) y E2E-10 (movilidad y costo-beneficio). E2E-3 y E2E-8 quedan como controles. Cada caso integral debe pedir al menos 2 intenciones de submódulos distintos, sin contar la L1 de SM1 que el orquestador agrega siempre.
- **Archivo o commit:** 2c11905† · `docs/plan/issues/PG3-11.md`†
- **Cómo se verificó:** Revisión cruzada del plan contra `ConsignaPG3.md` (nota `2026-11-10_Miranda_PG3-00_validacion-y-ajustes.md`†). **E2E-9 y E2E-10 todavía no se ejecutaron** (PG3-11).

C-10 · A1 · Plan (PG3-21) · Parcial

- **Problema detectado:** El pedido nuevo con inferencia en vivo estaba agendado después del congelamiento de código del jueves a las 22:00. Sin él, las reglas y el difuso corren solo al cargar el grafo, y la consigna §1 pide «realizar inferencias».
- **Corrección:** PG3-21 pasó a condicional, con una compuerta el miércoles: PG3-12 y PG3-13 en `main`, CP1 a CP11 en verde, Olea libre 5 h y ningún E2E roto. Si entra, termina el jueves a las 13:00; si no, se declara trabajo futuro (D-08). Se documentó qué código toca: `cypher/` con `$req`, `scripts/cargar_grafo.py` y el rechazo de nivel 2 en `router.validar`.
- **Archivo o commit:** 2c11905† · `docs/plan/PLAN_PG3.md`†
- **Cómo se verificó:** Revisión del código de PR #4: el contrato ya declara las intenciones de pedido nuevo y existe `Grafo.escribir`, así que no cambia el contrato. `tests/test_router.py::test_nivel_2_no_disponible` pasa hoy. **PG3-21 todavía no se ejecutó.**

C-11 · A1 · Plan (dependencias) · Corregida

- **Problema detectado:** Faltaban dependencias entre issues (PG3-11, PG3-14, PG3-15, PG3-18 y PG3-20), no había respaldo para el ítem l) del ZIP («Infografía o resumen ejecutivo»), `ARQUITECTURA.md` §7 a §9 estaba desactualizada y la numeración había saltado de PG3-04 a PG3-06.

- **Corrección:** Se agregaron las dependencias, la regla del ítem l) (si se recorta la infografía, se cubre con `docs/informe/02_resumen.md` en una página), `ARQUITECTURA.md` §7, §8 y §9 y el issue PG3-05, que ocupa el número saltado.
- **Archivo o commit:** 2c11905† · `docs/plan/` y `docs/ARQUITECTURA.md`†
- **Cómo se verificó:** Revisión cruzada del plan contra `ConsignaPG3.md` §4 y §7.

C-12 · A2 · Contextos · Parcial

- **Problema detectado:** Los nombres de archivo de los contextos eran dispares (`CONTEXTO_PROYECTO_SM1…`, `contexto_sm3_pg3.md` fuera de la carpeta) (S-01).
- **Corrección:** Se renombraron a `Contexto_SMX_{Apellido}_para_PG3.md`; el grupal quedó `Contexto_PG1_PG2_Grupo_para_PG3.md`.
- **Archivo o commit:** 75124b6
- **Cómo se verificó:** Listado actual de `docs/contexto/`: están los cinco nombres unificados. **Queda además un sexto archivo, copia anterior del de SM3** (ver C-20).

C-13 · A2 · Contexto PG1/PG2 · Corregida

- **Problema detectado:** El contexto citaba `CONTEXTO_PROYECTO_SM2_PARA_PG3.md`, que no existe (S-02).
- **Corrección:** La referencia apunta a `Contexto_SM2_Miranda_para_PG3.md`.
- **Archivo o commit:** 75124b6
- **Cómo se verificó:** Hoy no hay coincidencias de los nombres viejos (`CONTEXTO_PROYECTO_SM1/SM2`, `contexto_sm3_pg3`, `CONTEXTO_PG1_PG2_PARA_PG3`) en ningún `.md`, salvo la historia de INCONSISTENCIAS.

C-14 · A2 · Contexto de SM3 · Corregida

- **Problema detectado:** Las fórmulas LaTeX estaban rotas: `\t`, `\r` y `\f` se habían convertido en tabulación, retorno y salto de página (`imes`, `ightarrow`, `rac`) (S-04).
- **Corrección:** Se repararon 47 caracteres en el contexto de SM3.
- **Archivo o commit:** `docs/contexto/Contexto_SM3_deConinck_para_PG3.md` · 75124b6
- **Cómo se verificó:** Conteo hoy en el archivo vigente: 0 tabulaciones, 0 saltos de página y 0 «imes» sueltos. La copia vieja (C-20) conserva 24 tabulaciones, 1 salto de página y 5 «imes».

C-15 · A2 · Contextos y contrato · Corregida

- **Problema detectado:** Un mismo concepto tenía varios literales (D-06): `Descartada (excluyente)` y `Descartada (excluyente incumplido)`; `Indeterminada: completar información` con y sin tilde; escenarios `min`/`max` frente a `peor`/`mejor`; prioridad con y sin `grafica`; `GamaGPU` con `nivel` o `indice`.
- **Corrección:** Mandan los literales del canónico. Se reemplazaron los estados en PG1/PG2 (14), SM2 (1) y SM4 (3), y los escenarios de SM1 §3.4. Los literales quedaron en `asistente/contrato.py` (`ESTADOS`).
- **Archivo o commit:** 35a6e27 (contextos) · 1dee294 (`asistente/contrato.py`)
- **Cómo se verificó:** `tests/test_contrato.py::test_estados_de_la_evidencia_son_literales_d06`: todo `estado` de los fixtures con filas reales pertenece a `ESTADOS` (pasa hoy). CP1, CP2, CP4, CP9 y CP11 comparan los literales contra el grafo (11/11, no reproducido).

C-16 · A2 · Grafo (`cypher/`) · Corregida

- **Problema detectado:** Tres sentencias cargaban `estado_validacion = 'provisional'` (parámetros crisp, léxico de frecuencia y gamas de GPU), contra el literal `'sin validar'` de D-06.
- **Corrección:** Las tres pasaron a `'sin validar'` en `cypher/01_conocimiento.cypher`. El canónico histórico `docs/cypher/` no se toca: conserva las tres (líneas 353, 419 y 430 de `..._corregido.cypher`).
- **Archivo o commit:** 1dee294 (un solo commit; PG3-02 pedía commits separados)
- **Cómo se verificó:** Hoy no hay ningún `'provisional'` en `cypher/*.cypher`. La comparación de sentencias contra el canónico da 4 diferencias: estas tres y REC-R5 (fb5ce69†). `GRUPO.parametros` devuelve 77 elementos, todos `'sin validar'` (`tests/test_grupo.py`). `docs/informe/05_base_conocimiento.md` afirma que no queda ningún nodo `'provisional'`; la consulta de conteo no se reprodujo hoy.

C-17 · A2 · Contrato del orquestador · Corregida

- **Problema detectado:** Los submódulos usaban nombres de intención distintos: `explicar_notebook` con consultas diferentes en SM2 (L2) y SM4 (Q4); `fuera_de_alcance` en SM1 a SM3 y `desconocida` en SM4; `grafica`, `movilidad`, `mixta` y `modelo` en SM3. Además tenían tres formas de traza y de evidencia (D-07).
- **Corrección:** Lista cerrada de 13 intenciones con prefijo de submódulo, un plan, una evidencia y una traza comunes, en `asistente/contrato.py`.
- **Archivo o commit:** ce7dc44 (decisión) · 1dee294 (código)
- **Cómo se verificó:** `tests/test_contrato.py::test_esquema_plan_cerrado` (13 intenciones iguales al `enum` del esquema del plan), `test_forma_de_cada_fixture` y `test_hay_un_fixture_por_intencion_de_nivel_1` (pasan hoy).

C-18 · A2 · Numeración de casos · Parcial

- **Problema detectado:** `CP1` a `CP10` significaban cosas distintas en PG2, en el PI4 de SM1 y en el de SM4, y `E2E-1` a `E2E-n` tenían contenido distinto en cada contexto (D-12).
- **Corrección:** Prefijos por origen (`PG2-CPn`, `SM1-PI4-CPn`, `SM2-PI4-n`, `SM3-PI4-n`, `SM4-PI4-CPn`) y una sola lista oficial de E2E. **Residuo:** `scripts/verificar_grafo.py` y la sección 5 del informe siguen usando `CP1 a CP11` sin el prefijo `PG2-`.
- **Archivo o commit:** 28276b0
- **Cómo se verificó:** Revisión de lectura. Búsqueda de los prefijos en `asistente/`, `scripts/`, `tests/`, `prompts/` y `cypher/`: sin coincidencias.

C-19 · A2 · Contextos de SM3 y SM4 · Parcial

- **Problema detectado:** SM3 y SM4 no tenían etiquetas de procedencia, correcciones de la profesora ni discrepancias, y SM3 afirmaba estar «alineada 100 %» con un script que contradice a otros contextos (D-13).
- **Corrección:** Se agregó a ambos el anexo «Estado de unificación (D-13)» con la plantilla y las discrepancias. **Queda a cargo de cada autor** etiquetar sus datos y completar las correcciones de la profesora, que no están en ningún archivo del repositorio.
- **Archivo o commit:** 28276b0
- **Cómo se verificó:** Existencia del anexo: SM3, línea 340; SM4, línea 839. El etiquetado pendiente no se pudo verificar.

C-20 · A2 · `docs/contexto/CONTEXTO_SM3_PG3.md` · Abierta

- **Problema detectado: Detectado al preparar este registro.** Quedó en `docs/contexto/` una copia anterior del contexto de SM3 (22 918 bytes frente a 28 323 de la versión vigente). La dejó el commit ebc3c45 («renombrar y reubicar sm3») y conserva lo que S-01, S-04 y S-05 corrigieron en la otra: 24 tabulaciones, 1 salto de página, 5 «imes», los valores de movilidad anteriores (0,655 aparece dos veces) y «Alineada 100%». Ningún archivo la referencia.
- **Corrección:** Ninguna. Se propone eliminarla. No se tocó porque esta tarea es solo de documentación y el archivo es de de Coninck.
- **Archivo o commit:** ebc3c45
- **Cómo se verificó:** Conteo de caracteres y búsqueda de referencias en el repositorio (hoy).

C-21 · A3 · SM2 (consultas C1 y C2) · Corregida

- **Problema detectado:** Las consultas de PI4 de SM2 leían el riesgo desde `RAM/Notebook` (linaje A), y C1 del bloque 11 recalcula el excluyente de virtualización y **escribe** (`MERGE` de `INCUMPLE`).
- **Corrección:** C2 y C1 se portaron a `ev.*` (`Evaluacion`), de solo lectura; SM2 lee `ev.excluyente_virtualizacion`. `cypher/90_verificacion.cypher` no se carga, porque C1 escribe; los CP los corre `verificar_grafo.py` en modo lectura.
- **Archivo o commit:** `asistente/submodulos/sm2/repositorio.py` · 1dee294
- **Cómo se verificó:** `tests/test_sm2.py::ConGrafo::test_virtualizacion_coincide_con_c1` compara contra la CPU de cada notebook; `tests/test_grafo.py` comprueba que `Grafo.leer` no puede escribir. Corridas con base del 10/11 (88 tests), no reproducidas hoy.

C-22 · A3 · SM3 (consulta unificada y por modelo) · Corregida

- **Problema detectado:** La consulta de PI4 de SM3 traía las evaluaciones de todos los pedidos, sin

filtrar por el pedido.

- **Corrección:** Todas las consultas filtran por `{id:$requerimiento_id}`.
- **Archivo o commit:** `asistente/submodulos/sm3/repositorio.py` · 1dee294
- **Cómo se verificó:** `tests/test_sm3.py::test_filtra_por_el_pedido` (offline, pasa hoy) y `ConGrafo::test_solo_las_evaluaciones_del_pedido`: K está en `req_01` (VIRT) y en `req_03` (OFI), y cada pedido ve solo la suya.

C-23 · A3 · SM2 (filtro `tipo_ram`) · Corregida

- **Problema detectado:** La lista de tipos de RAM de PI4 era `ampliable`, `soldada` y `desconocida`. El grafo también tiene `mixta`, y seis de las nueve notebooks la tienen. El filtro no podía seleccionarla.
- **Corrección:** `TIPOS_RAM` incluye `mixta`.
- **Archivo o commit:** `asistente/contrato.py` · 1dee294; antes, `docs/fuentes/SM2/Miranda_Mauricio_PI4_U4.py`, línea 69
- **Cómo se verificó:** Los datos reales del grafo (`tests/fixtures/SM2.riesgo_ram.todas.json`: 1 ampliable, 6 mixta, 1 soldada, 1 desconocida) y la lectura del código. **No hay un test que filtre por `mixta`.**

C-24 · A3 · GRUPO (traza de reglas) · Parcial

- **Problema detectado:** `GRUPO.traza_reglas` pedía la `fuente` de cada `:Regla`, pero el canónico no la guarda: los 20 valores del fixture con filas reales eran `null`. El origen del conocimiento está en otros nodos (`Parametro.origen`, `EtiquetaDifusa.fuente`, `Notebook.fuente_precio`, `Bateria.fuente`).
- **Corrección:** Tres consultas nuevas (`origen_reglas`, `origen_notebook`, `origen_parametros`) y un bloque `origen` en la evidencia: los parámetros que respaldan cada regla, con su origen, su estado y la `ConsultaExperto` pendiente, y la fuente del precio y de la autonomía de la notebook. Lo que el grafo no tiene sale como `"no registrada"`; no se inventa.
- **Archivo o commit:** `asistente/submodulos/grupo/{repositorio,servicio,esquemas}.py` · fb5ce69†, 34f3c4d†
- **Cómo se verificó:** Contra Neo4j 5.26 (10/11): en la traza de la A, 14 de las 18 reglas tienen origen y 4 salen «no registrada» (GM-R3, INT-R11, REC-R4 y REC-R9); `tests/test_grupo.py` en verde. No reproducido hoy. **Límites:** el origen mostrado es el de los parámetros, no el del texto de la regla; solo se mira la variable del consecuente; las `ConsultaExperto` que validan una `:Regla` directamente no se devuelven.

C-25 · A3 · GRUPO (parámetros) · Corregida

- **Problema detectado:** `GRUPO.parametros` devolvía las 47 etiquetas y los 30 `Parametro` (77 en total) sin su `origen`.
- **Corrección:** Devuelve el `origen` de cada `Parametro`.
- **Archivo o commit:** `asistente/submodulos/grupo/` · fb5ce69†, 34f3c4d†
- **Cómo se verificó:** Contra Neo4j (10/11): los 77 elementos tienen origen y el estado de validación es `'sin validar'` en todos (nota de PG3-05; no reproducido hoy).

C-26 · A3 · SM3 (límite de resultados) · Corregida

- **Problema detectado:** Dos criterios de PG3-09 chocaban: «la X aparece con grado de GPU 0,74» y «orden descendente por adecuación gráfica». Con el límite de 5 resultados, la X (adecuación 0,844, la más baja de VIRT) queda afuera.
- **Corrección:** La prueba se hace con el filtro `dedicada: true`, donde aparecen C (0,902) y X (0,74). Sin filtro, los cinco primeros son B, A, M, E y K. Miranda aceptó el criterio con `dedicada` en la validación del plan del 10/11. La nota de PG3-09 preguntaba si convenía otro orden para «grafica»; en el repositorio no hay una respuesta de de Coninck a esa pregunta.
- **Archivo o commit:** `docs/plan/issues/PG3-09.md`† · 2c11905†
- **Cómo se verificó:** `tests/test_sm3.py::ConGrafo::test_x_grado_gpu_074` y los fixtures con filas reales: `SM3.buscar.json` (C 0,902 y 0,941; X 0,74 y 0,844) y `SM3.buscar.sin_filtros.json` (X ausente).

C-27 · A3 · SM2 (orden de las filas peor y mejor) · Abierta

- **Problema detectado:** Al regenerar los fixtures, el orden de las filas `peor` y `mejor` de `SM2.riesgo_ram` cambió entre corridas sin que cambiara el grafo. Según la lectura del código

(inferencia, no reproducida), se trata de las entradas de `reglas`: la consulta C2 las arma con `collect(DISTINCT …)` sin orden y su `ORDER BY` no incluye el escenario. La evidencia no se ve afectada, porque el servicio ordena por `(id, escenario)` (`sm2/servicio.py`, líneas 35 y 36); sí cambian las filas crudas de la traza y los fixtures.

- **Corrección: Ninguna.** Propuesta: agregar el escenario al ordenamiento de C2. Los dos fixtures de SM2 quedaron como estaban.
- **Archivo o commit:** `asistente/submodulos/sm2/repositorio.py` (C2) · nota de PG3-05†
- **Cómo se verificó:** Observado en una corrida real (nota de PG3-05) y confirmado por lectura del código; no reproducido hoy. Dueño: Miranda.

C-28 · A4 · SM4 (REC-R5) · Sin verificar

- **Problema detectado:** La prioridad `grafica`, que SM1 extrae, no sumaba peso: solo sumaban `rendimiento`, `portabilidad` y `autonomia`, aunque el enunciado de la regla dice «se suma a ese peso» (D-06).
- **Corrección:** En «Pesos del perfil», `peso_grafica` suma `P_INCREMENTO_PRIORIDAD` cuando `prioridad = 'grafica'`.
- **Archivo o commit:** `cypher/40_sm4.cypher` (línea 17) · 1dee294
- **Cómo se verificó:** Es una de las 4 sentencias que difieren del canónico (fb5ce69†), y CP1 a CP11 siguen igual. **Ningún pedido cargado tiene prioridad `grafica`** (`req_01` rendimiento, `req_02` ninguna, `req_03` portabilidad), así que nada ejercita el arreglo. Luconi debe revisar el cambio en su archivo.

C-29 · A4 · Grafo (REC-R3 y resultados por pedido) · Corregida

- **Problema detectado:** En el linaje A los resultados se guardaban en `Notebook`/`RAM` y REC-R3 contaminaba entre pedidos: dos compradores sobre la misma notebook se pisaban (corrección C-A, D-01 y D-11).
- **Corrección:** El canónico guarda los resultados en `Evaluacion` (pedido × notebook), y REC-R3 decide con `ev.excluyente_virtualizacion`.
- **Archivo o commit:** `cypher/15_evaluaciones.cypher`, `cypher/20_sm2.cypher`, `cypher/40_sm4.cypher` · 9080475 (decisión)
- **Cómo se verificó:** CP11: K queda `Descartada` en `req_01` y `Recomendada (Media)` 0,469 en `req_03` (11/11, no reproducido hoy). `tests/test_sm3.py::ConGrafo::test_solo_las_evaluaciones_del_pedido`. **La prueba de dos compradores sobre OFI (`req_04`, E2E-4) no existe todavía:** `req_04` no está cargado (el grafo tiene 3 requerimientos).

C-30 · A4 · SM4 (REC-R9 con prioridad «precio») · Sin verificar

- **Problema detectado:** La corrección de REC-R9 con prioridad «precio» (un `ORDER BY` nuevo, propuesto por SM4) no estaba aplicada en `bloques_0a12`.
- **Corrección:** Se incluyó en el canónico `corregido` y está en `cypher/40_sm4.cypher` (desde la línea 212).
- **Archivo o commit:** 9080475 (D-01) · 1dee294 (archivo dividido)
- **Cómo se verificó: Sin prueba.** La prevista era `req_05` (prioridad `precio`, puesto 1 = mayor costo-beneficio), que no existe en el grafo.

C-31 · A4 · Reglas (contextos frente al grafo) · Parcial

- **Problema detectado:** Los contextos daban por vigentes reglas que el grafo no tiene (REN-R11, REN-R13, REN-R14, REN-R2, GM-R1 y GM-R2, GM-R14, GM-R15, GM-R16, INT-R8 y INT-R9, entropía) y contradecían al grafo en otras: REN-R6 y REN-R9 están reemplazadas por FZ-R1 a FZ-R5, y REN-R10 decía «asumir no ampliable», contra la política de dato faltante (D-04).
- **Corrección:** Se alinearon los textos con el canónico: SM1 13-C, SM2 §5.1, SM3 §3 y PG1/PG2 §8.6 y §8.8. REN-R6 y REN-R9 figuran retiradas; REN-R10 conserva «desconocida». REC-R7, REC-R9 y REC-R13 quedaron resueltas con D-01. **La implementación de las demás se decidió y no se hizo** (ver Decisiones tomadas y no implementadas).
- **Archivo o commit:** 856872e
- **Cómo se verificó:** El grafo cargado tiene 57 reglas (23 + 27 + 7 retiradas). La alerta REN-R13 no existe: CP4 espera solo REN-R10 y FZ-R7, y SM2 devuelve dos alertas para la G (nota de PG3-09).

C-32 · A5 · SM3 (movilidad) · Corregida

- **Problema detectado:** Las movilidades del contexto de SM3 no se reproducían: A 0,655, B 0,914, E 0,655 y H 0,428 (S-05).
- **Corrección:** Se recalcularon con GM-RD1 a GM-RD3 y los parámetros del grafo: A 0,625, B 0,776, C 0,231, E 0,547 y H 0,430.
- **Archivo o commit:** `docs/contexto/Contexto_SM3_deConinck_para_PG3.md` · 75124b6
- **Cómo se verificó:** Coinciden con PG1/PG2 §9.3 y con la adecuación global de SM4 en Neo4j (INCONSISTENCIAS). Hoy, los fixtures con filas reales dan A 0,625, B 0,776, E 0,547, K 0,537 y X 0,486 (`SM3.buscar.sin_filtros.json`, `SM3.buscar.json`) y C 0,231 (CP6). **H (0,430) no figura en los fixtures** por el límite de 5: sin verificar en esta redacción.

C-33 · A5 · SM3 (grado de GPU) · Corregida

- **Problema detectado:** Los grados de GPU de X y de C en SM3 (0,55 y 0,916) no correspondían a ningún perfil.
- **Corrección:** Recalculados con GM-RD4 a GM-RD6: X 0,74 en VIRT, DIS y GAM y 0,472 en VID3D y DATOS; C 0,902. Coinciden con las adecuaciones gráficas de PG2 (0,6 × 0,74 + 0,4 = 0,844; 0,6 × 0,902 + 0,4 = 0,941).
- **Archivo o commit:** `docs/contexto/Contexto_SM3_deConinck_para_PG3.md` (§5) · 6b292eb
- **Cómo se verificó:** Los fixtures de `req_01` (VIRT) dan X 0,74 con adecuación 0,844 y C 0,902 con 0,941. El 0,472 (VID3D y DATOS) no se puede comprobar: ningún pedido cargado usa esos perfiles.

C-34 · A5 · SM1 (grado del perfil como rango) · Parcial

- **Problema detectado:** En PI4, SM1 ordenaba el perfil dominante con rango por el grado mínimo, y el CP1 de ese PI4 dio GAM (error). Un rango que cruza 0,4 no tenía tratamiento (D-09).
- **Corrección:** `L1` devuelve `grado_min` y `grado_max`, y el servicio informa el rango sin elegir uno. La decisión de evaluar los dos escenarios en el grafo **queda diferida**.
- **Archivo o commit:** `asistente/submodulos/sm1/` · 1dee294 (decisión: 28276b0)
- **Cómo se verificó:** `tests/test_sm1.py::test_rango_que_cruza_04_se_informa_como_rango`, con filas armadas a mano (pasa hoy). El canónico no guarda grados como rango (los tres pedidos tienen grados puntuales), así que no hay prueba contra Neo4j.

C-35 · A6 · LLM (proveedor, modelo y temperatura) · Corregida

- **Problema detectado:** SM1 usaba Groq (`openai/gpt-oss-120b`) con `langchain-openai`, que no están permitidos; SM2 usaba `llama3.1:8b`; SM3 y SM4 explicaban con temperatura 0,3 y 0,2 (D-05).
- **Corrección:** Un solo proveedor (Ollama local), un solo modelo por defecto y temperatura 0 en las dos llamadas. Se quitan `rich`, `python-dotenv`, `langchain-openai` y el paquete `ollama` suelto.
- **Archivo o commit:** c69ac45 (decisión) · 1dee294 (`asistente/llm.py`, `requirements.txt`)
- **Cómo se verificó:** `asistente/llm.py` fija `temperature=0`. `requirements.txt` solo lista `neo4j`, `langchain-core`, `langchain-ollama`, `requests`, `fastapi`, `uvicorn` y `httpx`. `tests/test_estructura.py` (pasa hoy) controla que LangChain solo aparezca en `llm.py` y en el orquestador y que no haya agentes.

C-36 · A6 · LLM (modelo por defecto) · Corregida

- **Problema detectado:** El modelo por defecto era `qwen2.5:7b`. Con entrega el 13/11, el `3b` corre en todas las notebooks del grupo (unos 2 GB en disco y 4 GB de RAM) y es de 2 a 3 veces más rápido (D-05, cambio del 10/11).
- **Corrección:** `qwen2.5:3b` por defecto y `qwen2.5:7b` opcional con `OLLAMA_MODEL`. Las debilidades del 3b (JSON del plan y redacción) se compensan con salida estructurada, validación en Python y verificador con plantilla fija.
- **Archivo o commit:** 7f93942 (decisión) · 1dee294 (`asistente/config.py`, `.env.example`)
- **Cómo se verificó:** Medición del router con `qwen2.5:3b` el 10/11: 24 de 25 intenciones y 25 de 25 resultados finales, en 217 s (nota de PG3-06; no reproducido hoy). **La latencia del plan, de 10 a 50 s en una máquina, no se midió en las demás.**

C-37 · A6 · LLM (cliente) · Corregida

- **Problema detectado:** El cliente era HTTP directo con `requests`, sin LangChain. El ejemplo de PG3 de la cátedra usa LangChain y la restricción original lo excluía (D-05, cambio del 10/11).

- **Corrección:** Cliente único en `asistente/llm.py` con `ChatOllama` y `ChatPromptTemplate`, y despacho a submódulos con `RunnableLambda` en el orquestador. `requests` queda solo para `GET /api/tags`. Sin agentes ni Cypher generado por LangChain. Excepción asentada en `docs/RESTRICCIONES.md` §8 (`langchain-ollama`, aprobada por el grupo).
- **Archivo o commit:** 1dee294 (`asistente/llm.py`, `asistente/orquestador.py`, `docs/RESTRICCIONES.md`)
- **Cómo se verificó:** `tests/test_estructura.py` (pasa hoy): LangChain solo en `llm.py` y el orquestador, sin agentes ni *tool calling*. `tests/test_orquestador.py::test_despacho_por_mapa_de_runnables` pasó en la corrida del 10/11 y no se pudo correr hoy (falta `langchain_core` en este entorno). **La confirmación de la excepción por Miranda sigue pendiente** (revisión del PR #4).

C-38 · A6 · Router (pedido) · Corregida

- **Problema detectado:** El LLM «completaba» `req_01` cuando el usuario no nombraba un pedido, y SM4 usaba `req_01` como pedido de sesión. SM2 ya había corregido ese error en PI4 (D-07 §3).
- **Corrección:** El pedido es el que escribió el usuario o el de la sesión; el que propone el LLM se descarta y queda anotado en `observaciones`; sin pedido se pregunta. Nunca se asume `req_01`.
- **Archivo o commit:** `asistente/router.py` (`validar`, paso 3) · 1dee294
- **Cómo se verificó:** `tests/test_router.py::test_pedido_inventado_por_el_llm_no_se_asume`, `test_pedido_de_la_sesion` y `test_manda_el_pedido_escrito`, y `tests/test_sm2.py::test_sin_pedido_no_usa_req_01` (pasan hoy). Medición del router con el LLM: 25 de 25 resultados finales (10/11).

C-39 · A6 · Orquestador (entrada) · Corregida

- **Problema detectado:** SM2 cortaba la entrada en 500 caracteres y SM4 la recortaba en 1500, en silencio (D-07 §8).
- **Corrección:** Se rechaza con una aclaración si está vacía, no tiene letras o supera 1500 caracteres. Nunca se recorta. La API responde con origen `validacion` y no con 422.
- **Archivo o commit:** `asistente/orquestador.py` · 1dee294
- **Cómo se verificó:** `tests/test_orquestador.py::test_1501_caracteres_no_se_recorta` y `test_vacia_y_sin_letras`; `tests/test_api.py::test_1501_caracteres_es_validacion_no_422`. Pasaron en la corrida del 10/11 (78 tests); hoy no se pudieron correr por dependencias faltantes.

C-40 · A6 · Router (fuera de alcance) · Corregida

- **Problema detectado:** En SM3, `fuera_de_alcance` derivaba a SM2, y SM4 lo llamaba `desconocida` (D-07).
- **Corrección:** El router decide: `fuera_de_alcance` devuelve un mensaje fijo y no ejecuta ninguna consulta al grafo.
- **Archivo o commit:** `asistente/router.py`, `asistente/orquestador.py` · 1dee294
- **Cómo se verificó:** `tests/test_orquestador.py::test_fuera_de_alcance_no_toca_el_grafo` (el grafo no recibe llamadas) y `tests/test_router.py::test_fuera_de_alcance` (pasa hoy).

C-41 · A6 · Router (guarda de dominio) · Corregida

- **Problema detectado:** Con `qwen2.5:3b`, el LLM marcaba `fuera_de_alcance` en consultas del dominio y, al revés, planificó «la capital de Francia» como `GRUPO.parametros` (consulta R18 del CSV de pruebas).
- **Corrección:** `validar` ignora la marca si el texto trae palabras del dominio (lista `_DOMINIO`) y trata como fuera de alcance un plan sin consultas cuando no las trae.
- **Archivo o commit:** `asistente/router.py` · 1dee294
- **Cómo se verificó:** `tests/test_router.py::test_fuera_de_alcance_falso_se_ignora` (pasa hoy); medición del 10/11 sobre 25 consultas: 25 de 25 resultados finales, con 24 de 25 intenciones antes de la validación (nota de PG3-06; no reproducido hoy).

C-42 · A6 · Orquestador (advertencia) · Corregida

- **Problema detectado:** La advertencia de parámetros sin validar se omitía en `virtualizacion` (SM2) y se ponía siempre en SM4 (D-07 §7).
- **Corrección:** Se agrega siempre que se consulta el grafo (`evidencia["advertencia"]`), porque todo

está `'sin validar'`, incluida REN-R1.

- **Archivo o commit:** `asistente/orquestador.py` (línea 117) · 1dee294
- **Cómo se verificó:** `tests/test_orquestador.py::test_punta_a_punta_con_stubs` comprueba `ev["advertencia"] == ADVERTENCIA` (corrida del 10/11; no se pudo correr hoy).

C-43 · A6 · Evidencia común (alertas) · Corregida

- **Problema detectado:** La verificación de «atribución de cada alerta a su notebook» había quedado pendiente en SM2 (D-07 §6): una alerta sin notebook podía atribuirse a otra.
- **Corrección:** Toda alerta de la evidencia lleva `notebook` (`codigo`, `texto`, `notebook`).
- **Archivo o commit:** `asistente/submodulos/sm2/servicio.py` y demás servicios · 1dee294
- **Cómo se verificó:** `tests/test_contrato.py::test_alertas_con_notebook` recorre los fixtures con filas reales (pasa hoy). **El verificador común que controla la atribución (PG3-13) todavía es un stub.**

Verificado sin cambios

Revisiones y corridas que no encontraron nada que corregir. Se agregan para que cada aspecto tenga una evidencia explícita.

V-01 · A1

- **Qué se verificó:** Que el grafo dividido en `cypher/` sea el canónico entero, salvo los arreglos intencionales de D-06.
- **Evidencia:** Sin Neo4j: 101 sentencias contra 101; difieren 4 (C-16 y C-28) y las otras 97 son idénticas (`python -m scripts.contar_grafo --sentencias`, `tests/test_contar_grafo.py`†). Con Neo4j 5.26 (10/11): 343 nodos y 683 relaciones en ambos, y coinciden las 57 filas (28 labels y 29 tipos de relación) (`docs/pruebas/conteos_grafo.md`†, 34f3c4d†). No reproducido hoy: el script está en la rama †.
- **Resultado:** Sin cambios

V-02 · A3

- **Qué se verificó:** Regresión de las consultas sobre el grafo cargado y los tests con base.
- **Evidencia:** `python -m scripts.verificar_grafo`: CP1 a CP11 en 11/11 (notas de PG3-01 y PG3-05). Con la base, 88 tests pasan y 1 se saltea (router con Ollama) (nota de PG3-05). Hoy, sin Neo4j, los módulos de test que no importan LangChain dan 64 tests OK y 12 omitidos.
- **Resultado:** Sin cambios

V-03 · A4

- **Qué se verificó:** Reglas con efecto en el estado de las notebooks.
- **Evidencia:** CP2 (REN-R1 y REC-R3: la E queda `Descartada (excluyente incumplido)` pese a 0,81), CP7 (desempate por ampliabilidad: A, H, B), CP9 (REC-R15: M `Recomendada condicionada por falta de informacion`, grado [0,504 ; 0,826]) y CP10 (`req_02` perfil indeterminado, sin evaluaciones y sin asumir OFI). `tests/test_grupo.py::ConGrafo::test_traza_a`: la traza de la A reúne reglas de SM1 a SM4 (20 filas). Nota de PG3-09.
- **Resultado:** Sin cambios

V-04 · A5

- **Qué se verificó:** Valores de los componentes difusos.
- **Evidencia:** CP4 (riesgo de la G [49,58 ; 81,46]), CP5 (K 49,55 medio), CP6 (movilidad de la C 0,231), CP8 (47 etiquetas `'sin validar'` en 15 variables) y CP9. Revisión de PG1/PG2 §15.10: la división por `null` no aplica al canónico, porque ρ usa `ram_min` del perfil dominante (INCONSISTENCIAS §3).
- **Resultado:** Sin cambios

V-05 · A6

- **Qué se verificó:** Que el router enrute bien y que la integración respete «Neo4j maneja la verdad».
- **Evidencia:** Router con `qwen2.5:3b` el 10/11: 24 de 25 intenciones (96 %) y 25 de 25 resultados finales, en 217 s (`PG3_TEST_LLM=1 python -m unittest tests.test_router`; no reproducido hoy: sin Ollama). Tests offline de `validar` y de la estructura (el LLM no escribe Cypher, LangChain solo

en `llm.py` y el orquestador, `Grafo.leer` no puede escribir): pasan hoy en los módulos que se pudieron importar.
- **Resultado:** Sin cambios

Decisiones tomadas y no implementadas

No son correcciones realizadas: son decisiones de INCONSISTENCIAS cuyo cambio en el código o en los datos todavía no existe. El informe debe presentarlas como trabajo futuro si no se llega a hacerlas (`docs/plan/PLAN_PG3.md` §1). La columna «Issue» sale de la tabla de issues de ese plan.

Decisión	Qué falta	Issue	Filas relacionadas
D-01	Portar al canónico las piezas de SM1 del linaje A	PG3-21	C-03
D-03	Cargar `SALIDA_PI2` de los 7 perfiles (53 filas pendientes más K en OFI)	PG3-25	C-06
D-04	REN-R2, REN-R11, REN-R13 y REN-R14	PG3-23	C-31
D-04	GM-R1, GM-R2, GM-R14, GM-R15 y GM-R16	PG3-24	C-31
D-04	INT-R8, INT-R9 y entropía	PG3-22	C-31
D-04	REC-R16	PG3-26	C-31
D-08	Pedido nuevo: registrar y aclarar, con inferencia por `$req` (condicional a la compuerta)	PG3-21	C-10
D-09	Evaluar los dos escenarios cuando el perfil o la intensidad son inciertos	PG3-22	C-34
D-10	`requisitos_explicitos`	PG3-22	C-07
D-11	`req_04` y `req_05`, y el E2E-4 de dos compradores	PG3-25 y PG3-26	C-29, C-30
D-13	Etiquetas de procedencia en SM3 y SM4	PG3-27	C-19

Pendiente de agregar

- **Bugs de los E2E (PG3-11).** Cada E2E que falle abre un issue al dueño del submódulo y suma una fila a este registro. A la fecha de redacción no se ejecutó ninguno.
- **Revisión del PR #4** (PG3-05, a cargo de Miranda). Tres puntos: aprobar `asistente/contrato.py`, confirmar la excepción de `langchain-ollama` en `docs/RESTRICCIONES.md` §8 (C-37) y confirmar los cambios de `CLAUDE.md` en las reglas «Contrato» y «LLM». La validación del plan también anota que el PR #4 se abrió y se unió por la misma persona, sin revisión; ese hallazgo de proceso va en la revisión.
- **SM4, explicador y verificador (PG3-12 y PG3-13).** Siguen como stub. Cuando existan hay que registrar su verificación: tasa de plantilla fija, verificador semántico (rangos, perfil indeterminado, alertas) y voseo de la explicación (D-07 §5 y §6).
- **Latencia.** El plan tarda de 10 a 50 s con `qwen2.5:3b` en una máquina, y el explicador suma otra llamada. Falta medir en las demás (C-36).
- **Cambios de PG3-05 sin unir a `main`.** Las filas marcadas con † cambian de hash al unirse.
- **Seguimientos menores de PG3-05.** Quitar la columna `q.fuente` de L3 (siempre `null`) y mover `NO_REGISTRADA` a `contrato.py`, con el visto bueno de Miranda.
- **Filas abiertas.** C-20 (copia vieja del contexto de SM3) y C-27 (orden de peor y mejor en SM2). Revisión de Luconi del cambio en `cypher/40_sm4.cypher` (C-28).

Límites de este registro

- **Qué se corrió al redactarlo (10/11/2026):** los módulos de test que no importan LangChain (`tests.test_router`, `test_contrato`, `test_sm1`, `test_sm2`, `test_sm3`, `test_grupo` y `test_estructura`): **64 tests OK y 12 omitidos** (los que necesitan Neo4j u Ollama). Los módulos `test_orquestador` y `test_api` no se pudieron importar porque el entorno no tiene `langchain_core`; no se instaló nada.
- **Qué no se reprodujo:** todo lo que requiere Neo4j u Ollama (CP1 a CP11, conteos, tests `ConGrafo`, medición del router, regeneración de fixtures). Esos datos se citan de las notas de sesión de PG3-01, PG3-05, PG3-06, PG3-09 y de la validación del plan, y cada fila lo indica.
- **Qué se contrastó directamente con el repositorio:** los valores de movilidad y de grado de GPU

contra los fixtures con filas reales, la ausencia de `'provisional'` en `cypher/`, las prioridades de los tres pedidos, los nombres de los archivos de `docs/contexto/`, y el contenido de las ramas `pg3-05-cierre-trazabilidad` y `docs/validacion-plan-pg3` con `git show`.

- **Evidencia por commit:** el código de PG3-01 a PG3-09 llegó en un único commit (1dee294, PR #4), de modo que el hash no distingue entre correcciones; la distinción está en el archivo citado.

Anexo B · Manual de uso

> **Para quién:** para alguien que no es del grupo (por ejemplo, el docente) y quiere instalar el asistente, cargar su base de conocimiento y hacer una consulta sin ayuda. Es el ítem d) de la entrega (consigna §4). **Sistema operativo:** los comandos son de **PowerShell en Windows**, que es lo que usa el grupo. Las diferencias para Linux y macOS están en la sección 11. **Probado con:** Windows 11, Python 3.14, Ollama con `qwen2.5:3b` y, por el grupo, Neo4j 5.26 local (ver `docs/pruebas/conteos_grafo.md`).

Índice

1. Requisitos
2. Instalación
3. Cargar y verificar el grafo
4. Usar el asistente
5. Consultas de ejemplo
6. Cómo leer una respuesta
7. Ver la traza
8. Errores comunes
9. Probar el sistema
10. Dónde seguir
11. Linux y macOS

De cero a la primera respuesta, en una mirada

Paso	Qué	Sección
1	Instalar Python, Ollama y Neo4j 5	1 y 2
2	Crear el entorno virtual e instalar `requirements.txt`	2.2
3	Descargar el modelo: `ollama pull qwen2.5:3b`	2.3
4	Copiar `.env.example` a `.env` y poner la contraseña de Neo4j	2.5
5	Cargar el grafo y comprobar que da `11/11 OK`	3
6	Hacer una consulta con la consola o con la API	4 y 5

---

1. Requisitos

Hardware

Recurso	Mínimo	Nota
Memoria RAM	~4 GB libres para el modelo `qwen2.5:3b` (~8 GB si usás `qwen2.5:7b`)	Más lo que consuma Neo4j
Disco	~2 GB para el modelo (`qwen2.5:3b` pesa 1,9 GB) más el espacio de Neo4j y de las dependencias de Python	
Red	Solo para instalar (paquetes de Python y descarga del modelo)	Después el sistema corre local, sin APIs pagas

El proyecto no exige placa de video. La velocidad de las respuestas depende del equipo (ver Tiempos).

Software

Herramienta	Versión	Para qué
Windows + PowerShell	10 u 11	Los comandos de este manual
Python	3.10 o posterior (probado con 3.14)	El código del asistente
Ollama	Reciente, con el modelo `qwen2.5:3b`	El LLM local: interpreta la consulta y redacta la respuesta

Neo4j	5.x (probado con 5.26): Neo4j Desktop local o AuraDB Free	La base de conocimiento, que es la que maneja la verdad
Git	Opcional	Solo si bajás el código con `git clone`

Puertos que usa

Puerto	Servicio
7687	Neo4j (protocolo Bolt)
7474	Neo4j Browser, solo si querés mirar el grafo (opcional)
11434	Ollama
8000	La API del asistente (se puede cambiar con `--port`)

---

2. Instalación

2.1 Obtener el código

Con el ZIP de la entrega, descomprimilo. Con Git:

```
git clone <URL del repositorio>
```

Abrí PowerShell **en la carpeta raíz del proyecto**, la que tiene `requirements.txt`. Todos los comandos de este manual se corren desde ahí.

2.2 Python: entorno virtual y dependencias

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Instala solo lo que está en `requirements.txt`: `neo4j`, `langchain-core`, `langchain-ollama`, `requests`, `fastapi`, `uvicorn` y `httpx`. Si PowerShell dice que la ejecución de scripts está deshabilitada, corré esto y repetí la activación (solo vale para esa ventana):

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Si no tenés Python, instalalo desde python.org y marcá **Add python.exe to PATH**.

2.3 Ollama y el modelo

1. Instalá Ollama desde https://ollama.com/download.
2. Descargá el modelo (1,9 GB):

```
ollama pull qwen2.5:3b
ollama list
```

`ollama list` tiene que mostrar `qwen2.5:3b`. Si da un error de conexión, abrí la aplicación Ollama o corré `ollama serve` en otra terminal.

> Opcional: `qwen2.5:7b` da respuestas algo mejores pero pide ~8 GB de RAM. Para usarlo, bajalo con `ollama pull qwen2.5:7b` y poné `OLLAMA_MODEL=qwen2.5:7b` en el `.env`. El proyecto está medido con `qwen2.5:3b`.

2.4 Neo4j

Hace falta una base **vacía y dedicada a este proyecto**: el paso 3 la **borra** antes de cargar el grafo.

Con Neo4j Desktop (local): creá una instancia nueva con una versión 5.x, elegí una contraseña, iniciala y dejá el puerto Bolt por defecto (7687). Los nombres de los menús cambian entre versiones de Desktop; lo que importa es tener la instancia corriendo y la contraseña a mano.

Con AuraDB Free: creá una instancia y guardá el usuario, la contraseña y la URI que te da Aura (empieza con `neo4j+s://`). Esa URI va en `NEO4J_URI`. No se probó el flujo completo con Aura; el grupo trabajó con Neo4j local.

2.5 El archivo `.env`

Copiá la plantilla y completá la contraseña de Neo4j:

```
Copy-Item .env.example .env
notepad .env
```

Variable	Qué es	Valor por defecto
NEO4J_URI	Dónde está Neo4j	bolt://localhost:7687
NEO4J_USER	Usuario de Neo4j	neo4j
NEO4J_PASSWORD	**La contraseña que elegiste.** Obligatoria	(vacía)
NEO4J_DATABASE	Base dentro de Neo4j	neo4j
OLLAMA_URL	Dónde está Ollama	http://localhost:11434
OLLAMA_MODEL	Modelo de Ollama	qwen2.5:3b
MAX_CARACTERES	Largo máximo de una consulta	1500
MAX_REGENERACIONES	Veces que el explicador reintenta antes de usar la plantilla fija	2
DIR_TRAZAS	Carpeta donde se guarda la traza	trazas

Escribí los valores sin comillas. **El `.env` no se sube nunca a Git** (está en `.gitignore`: el repositorio es público) y no se comparte.

El código **no lee el `.env` solo**: las variables se cargan en la ventana de PowerShell antes de lanzar cualquier comando. Es el primer paso del bloque de la sección 4.1.

---

3. Cargar y verificar el grafo

Con la terminal preparada como en la sección 4.1 y Neo4j iniciado:

```
python -m scripts.cargar_grafo
```

Borra todo lo que haya en esa base y carga los scripts de `cypher/` en orden. Tiene que terminar así:

```
Cargando el grafo en bolt://localhost:7687 ... (BORRA todo lo que haya en la base)
  00_esquema.cypher: 12 sentencias
  01_conocimiento.cypher: 16 sentencias
  02_instancias.cypher: 27 sentencias
  10_sm1.cypher: 7 sentencias
  15_evaluaciones.cypher: 1 sentencias
  20_sm2.cypher: 11 sentencias
  30_sm3.cypher: 9 sentencias
  40_sm4.cypher: 18 sentencias
Listo: 101 sentencias.
```

Después, la verificación:

```
python -m scripts.verificar_grafo
```

Compara lo que devuelve el grafo con los 11 casos de prueba de la base (CP1 a CP11) y no escribe nada. Tiene que terminar con **`11/11 OK`**. Si algún caso falla, la última línea lista cuáles y el comando sale con código 1: volvé a cargar el grafo.

---

4. Usar el asistente

Hay dos formas de usarlo y hacen lo mismo: la **consola** (la más simple) y la **API**. Las dos pasan por el mismo camino: el router interpreta la consulta, el sistema consulta Neo4j con consultas Cypher fijas de solo lectura, y el explicador redacta la respuesta a partir de lo que devolvió el grafo.

4.1 Cada vez que abrís una terminal

Las variables de entorno y el entorno virtual valen **solo para la ventana donde los cargaste**. En cada ventana nueva, desde la carpeta raíz del proyecto:

```
cd C:\ruta\al\proyecto
.venv\Scripts\Activate.ps1
Get-Content .env | Where-Object { $_ -match '^\s*[^#\s][^=]*=' } | ForEach-Object {
    $nombre, $valor = $_ -split '=', 2
```

```
    Set-Item -Path "Env:$($nombre.Trim())" -Value $valor.Trim()
}
```

Con eso quedan activos el entorno de Python y las variables del `.env`. Ollama y Neo4j tienen que estar corriendo.

4.2 La consola

Una sola consulta:

```
python -m asistente.cli "Para req_01, ¿qué notebook me recomendás? ¿Cómo andan de RAM y de gráfica?"
```

Sale la respuesta y, debajo, una línea con el origen, el pedido y los submódulos consultados:

```
<respuesta del asistente>

  [origen: llm · pedido: req_01 · submódulos: SM1, SM2, SM3, SM4]
```

Opciones (todas son de `asistente/cli.py`):

Opción	Qué hace
`--traza`	Agrega debajo de la respuesta el plan, las consultas ejecutadas, la duración y el error si lo hubo (ver sección 7)
`--pedido req_01`	Fija el pedido de la sesión, para no escribirlo en cada consulta
(sin texto)	Abre el modo conversación

Modo conversación:

```
python -m asistente.cli
```

```
[sin pedido] > /pedido req_01
Pedido de la sesión: req_01

[req_01] > ¿Qué me recomendás?
```

Comandos del modo conversación: `/pedido req_01` fija el pedido y `/salir` termina (también `/exit`, `/q`, Ctrl+C o Ctrl+Z y Enter). Cualquier otra línea es una consulta. En la consola, el pedido de la sesión queda guardado mientras no salgas.

4.3 La API

```
uvicorn asistente.api:app
```

Queda escuchando en `http://127.0.0.1:8000` (si el puerto está ocupado: `uvicorn asistente.api:app --port 8001`). Se corta con Ctrl+C. Es la entrada única del sistema y tiene dos operaciones:

Operación	Para qué
`GET /salud`	Dice si Neo4j y Ollama (con el modelo descargado) responden. `ok: true` solo si ambos están bien
`POST /consultar`	Hace una consulta. Entrada: `texto` (obligatorio, hasta 1500 caracteres) y `requerimiento_id` (opcional, por ejemplo `req_01`)

Desde el navegador (lo más simple): abrí `http://localhost:8000/docs`, elegí `POST /consultar`, **Try it out**, completá el cuerpo y **Execute**. Esa página carga sus componentes desde internet.

Desde PowerShell. Pegá una vez esta función y usala (los bytes en UTF-8 evitan que se rompan las tildes en Windows PowerShell 5.1):

```
function Consultar($texto, $pedido) {
    $c = @{ texto = $texto }
    if ($pedido) { $c.requerimiento_id = $pedido }
    $bytes = [System.Text.Encoding]::UTF8.GetBytes(($c | ConvertTo-Json))
    $r = Invoke-WebRequest -Uri http://localhost:8000/consultar -Method Post -ContentType "application/
json; charset=utf-8" -Body $bytes -UseBasicParsing
    [System.Text.Encoding]::UTF8.GetString($r.RawContentStream.ToArray()) | ConvertFrom-Json
}

$res = Consultar "Para req_01, ¿qué notebook me recomendás? ¿Cómo andan de RAM y de gráfica?" "req_01"
$res.respuesta
```

```
$res | Select-Object requerimiento_id, origen_respuesta, intento, submodulos_consultados
```

Para ver la salud:

```
(Invoke-WebRequest http://localhost:8000/salud -UseBasicParsing).Content
```

La respuesta de `POST /consultar` trae estos campos:

Campo	Qué es
`respuesta`	El texto para el asesor
`requerimiento_id`	El pedido sobre el que se respondió (`null` si la consulta no usó ninguno)
`submodulos_consultados`	Qué submódulos se consultaron (`SM1` a `SM4` y `GRUPO`)
`origen_respuesta`	De dónde salió la respuesta (ver sección 6)
`intento`	Intento del explicador: `1` es a la primera, `2` o `3` indica que hubo que regenerar, `0` que no se llegó a explicar
`traza`	El registro completo de la consulta (ver sección 7)

> **Cada llamada a la API es independiente:** no recuerda el pedido de la anterior. Escribí el pedido en el texto (`req_01`) o mandá `requerimiento_id` en cada consulta.

Tiempos

Cada consulta hace dos llamadas al modelo (una para interpretar y otra para redactar) y puede reintentar la redacción. Con `qwen2.5:3b`, en la máquina de de Coninck el plan solo tardó entre 10 y 50 s (`PLAN_PG3.md` §5), así que una consulta completa puede llevar **de varios segundos a más de un minuto**. La primera es la más lenta porque Ollama carga el modelo en memoria. El límite por llamada es de 120 s: pasado ese tiempo, la respuesta es el error del modelo (sección 8).

---

5. Consultas de ejemplo

Son los casos integrales de las pruebas E2E (de punta a punta) y el control de fuera de alcance. **Lo que decide si una respuesta está bien es lo que dice el grafo** (estados, grados, puestos, alertas), no la redacción exacta del modelo, que puede variar entre corridas y se limita a unas 150 palabras. Para cada consulta se indica qué tiene que aparecer.

Corré cada una desde la terminal preparada (sección 4.1):

```
python -m asistente.cli "Para req_01, ¿qué notebook me recomendás? ¿Cómo andan de RAM y de gráfica?"
python -m asistente.cli "En req_01, ¿por qué la M quedó condicionada y la G indeterminada?"
python -m asistente.cli "En req_01, la E tiene casi el mismo puntaje que la B y la H, ¿por qué no la
recomiendan?"
python -m asistente.cli "En req_01, ¿qué reglas se activaron para la A, de dónde sale cada dato y qué
riesgo de RAM tiene?"
python -m asistente.cli "Para req_01, ¿cuáles son las mejores en movilidad y cómo queda el costo-
beneficio?"
python -m asistente.cli "¿Cuál es la capital de Francia?"
```

Caso	Qué muestra	Submódulos	Qué tiene que decir el grafo
E2E-1	Una recomendación completa que cruza cuatro submódulos	SM1, SM2, SM3 y SM4	`Notebook_A` en el puesto 1, `Recomendada (Alta)`, grado 0,785. Después `Notebook_H` (puesto 2) y `Notebook_B` (puesto 3), ambas `Recomendada (Media)`. `Notebook_X` aparece aparte, como alternativa **fuera de presupuesto** (+5 %). `Notebook_M` es condicionada, `Notebook_G` indeterminada, `Notebook_E` y `Notebook_C` descartadas. La respuesta puede no nombrarlas todas: el ranking completo está en la traza
E2E-2	Un dato faltante que atraviesa módulos	SM2, SM3 y SM4	`Notebook_M`: `Recomendada condicionada por falta de informacion`, grado entre 0,504 y 0,826, porque la ficha no informa la autonomía (REC-R15). `Notebook_G`: `Indeterminada: completar informacion`, riesgo de RAM entre 49,58 y 81,46, con las alertas REN-R10 y FZ-R7
E2E-5	Un requisito excluyente que pesa más que el puntaje	SM2 y SM4	`Notebook_E` (adecuación 0,81) está casi igual que `Notebook_B` (0,813) y `Notebook_H` (0,811), pero es `Descartada (excluyente incumplido)`: su CPU no soporta virtualización, que el perfil exige (REN-R1 y REC-R3)

E2E-9	Trazabilidad de una notebook	GRUPO, SM2 y SM4	La traza de reglas de `Notebook_A` (18 reglas distintas de los cuatro submódulos, cada una con su fuerza), de dónde sale cada dato (14 de las 18 reglas tienen origen; GM-R3, INT-R11, REC-R4 y REC-R9 figuran como «no registrada»), la fuente del precio y de la autonomía, y el riesgo de RAM tal como lo devuelve el grafo
E2E-10	Búsqueda por movilidad más costo-beneficio	SM3 y SM4	Notebooks ordenadas de mayor a menor adecuación de movilidad. Si una no tiene el dato (por ejemplo `Notebook_M`, sin autonomía informada) se dice que falta, **sin inventar un número**. El costo-beneficio sale del grafo: por ejemplo, `Notebook_B` tiene 1,161
E2E-8 (control)	Una pregunta que no es del dominio	Ninguno	`origen: fuera_de_alcance`, el mensaje fijo de alcance y **ninguna consulta al grafo** (en la traza, `consultas` queda vacía). Necesita igual Neo4j y Ollama levantados: el sistema lee el catálogo y el router (que usa el modelo) decide que no es del dominio

Un control más, para ver cómo se lee un pedido sin perfil definido:

```
python -m asistente.cli "Para req_02, ¿qué notebook me conviene?"
```

`req_02` tiene el perfil `indeterminado`: el sistema **no asume** ningún perfil, no evalúa ninguna notebook y repite la pregunta pendiente («¿Qué programas o actividades va a usar con la notebook?»).

Los submódulos de la columna son los que tiene que consultar el router; la línea `[origen: … · submódulos: …]` de la consola muestra los que realmente se consultaron. `SM1` aparece siempre que hay un pedido, porque el sistema lo agrega para llevar el perfil y el presupuesto en la evidencia.

Para controlar una respuesta contra el grafo

Si querés comprobar lo que dijo el asistente por tu cuenta, abrí Neo4j Browser (`http://localhost:7474`, o el botón *Open* de Neo4j Desktop) y corré esta consulta de solo lectura, que tiene la misma forma que las de `scripts/verificar_grafo.py`:

```
MATCH (:Requerimiento {id:'req_01'})-[:EVALUA]->(ev:Evaluacion)-[:SOBRE]->(n:Notebook)
RETURN n.modelo AS notebook, ev.estado AS estado, ev.grado_recomendacion AS grado,
       ev.grado_recomendacion_min AS grado_min, ev.grado_recomendacion_max AS grado_max
ORDER BY grado DESC
```

---

6. Cómo leer una respuesta

Una respuesta tiene cuatro partes: el **texto**, el **origen**, los **estados y rangos** que cita y la **advertencia** del final.

`origen_respuesta`: de dónde salió el texto

En la consola aparece como `[origen: …]`; en la API es el campo `origen_respuesta`.

Valor	Qué significa	Qué hacer
`llm`	La redactó el modelo y el **verificador** la aprobó: cada cifra, notebook, regla y alerta que cita está en lo que devolvió el grafo	Nada: es el caso normal
`plantilla_fija`	El modelo no logró una redacción que pasara el verificador (se intenta 1 vez más 2 reintentos). Se muestra un texto armado por código con los datos del grafo	Es correcta pero seca (líneas como `SM4.recomendar:` y los datos). Podés reformular la consulta
`validacion`	La entrada no se pudo procesar y el asistente **pide una aclaración**: consulta vacía o muy larga, pedido o modelo que no existe, falta el pedido	Leé el mensaje: dice qué falta (sección 8)
`fuera_de_alcance`	La consulta no es del dominio. Mensaje fijo, sin tocar el grafo	Preguntá sobre notebooks, pedidos o reglas del sistema
`error`	Neo4j u Ollama no respondieron, o falló algo inesperado. Mensaje fijo	Sección 8; el detalle técnico está en la traza (`error`)

Estados de una notebook

Son siempre estos seis, escritos así (sin tildes, como están en el grafo). Los decide la base de conocimiento, no el modelo:

Estado	Qué significa para el asesor	Ejemplo en `req_01`

`Recomendada (Alta)`	Cumple lo que pide el perfil y su grado de recomendación es alto: opción firme	`Notebook_A`, grado 0,785
`Recomendada (Media)`	Cumple, con un grado medio: es para considerar	`Notebook_H` y `Notebook_B`
`Recomendada condicionada por falta de informacion`	Podría recomendarse, pero a la ficha le falta un dato: el grado sale como **rango** y hay que completar el dato	`Notebook_M` (falta la autonomía)
`Indeterminada: completar informacion`	Falta un dato clave y **no se puede decidir**	`Notebook_G` (no se sabe si su RAM es ampliable)
`Descartada (excluyente incumplido)`	No cumple un requisito que el perfil exige, aunque su puntaje sea parecido al de las recomendadas	`Notebook_E` (su CPU no soporta virtualización)
`Descartada`	Su grado de recomendación quedó bajo	`Notebook_K` (8 GB de RAM contra 16 GB mínimos)

Una notebook que se pasa un poco del presupuesto no se recomienda en primer lugar: aparece aparte, como **alternativa fuera de presupuesto**, con el porcentaje que se pasa.

Rangos, «desconocida» e «indeterminado»

El sistema **no completa ni promedia** lo que no sabe:

- **Rango:** si falta un dato, el valor se informa como un intervalo, por ejemplo «el riesgo de RAM de la `Notebook_G` está entre 49,58 y 81,46 sobre 100». Son el peor y el mejor caso; no hay un número «del medio». El grado de `Notebook_M`, entre 0,504 y 0,826, se lee igual.
- **`desconocida`:** el dato no figura en la ficha (por ejemplo, si la RAM de la G es ampliable). No equivale a «no» ni a «sí».
- **`indeterminado`:** un pedido sin perfil definido (como `req_02`). No se elige un perfil por el comprador.

Alertas y reglas

Los códigos como `REN-R10`, `FZ-R7` o `REC-R15` son las reglas de la base de conocimiento. El prefijo dice de qué submódulo vienen: `INT` (SM1, interpretación del pedido), `REN` y `FZ` (SM2, rendimiento: reglas crisp y difusas), `GM` (SM3, gráfica y movilidad) y `REC` (SM4, recomendación). Una **alerta** es un aviso del grafo sobre **una notebook concreta** («la ficha no informa si la RAM es ampliable»); no es lo mismo que una regla activada. Una alerta siempre se cita junto con su notebook.

La advertencia del final

Toda respuesta que consulta el grafo cierra con este texto, copiado literal:

> Los parametros difusos y los umbrales del sistema estan sin validar por el tecnico: los grados y riesgos son una referencia, no una certeza.

Es una limitación real: los parámetros del modelo difuso (centros, sigmas y umbrales) todavía **no los validó un técnico**. Los grados y los riesgos sirven para ordenar y comparar, no como un dato cierto. Si el modelo se olvida de copiarla, la agrega el código.

---

7. Ver la traza

Cada consulta deja un registro que permite **reconstruir cómo se llegó a la respuesta**: qué entendió el modelo, qué se le preguntó a Neo4j, qué datos volvieron y qué se le pasó al explicador. Hay tres formas de verlo.

1. En la consola, con `--traza`:

```
python -m asistente.cli --traza "Para req_01, ¿qué notebook me recomendás?"
```

Debajo de la respuesta imprime el plan (`plan:`), las observaciones de la validación, una línea por consulta ejecutada (nombre, milisegundos, cantidad de filas y parámetros), la duración y, si hubo, el `error:`.

2. En la API: es el campo `traza` de la respuesta (`$res.traza` con la función de la sección 4.3).

3. En el archivo: una línea JSON por consulta en `trazas\AAAA-MM-DD.jsonl`, dentro de la carpeta desde donde lanzaste el asistente (o la de `DIR_TRAZAS`). Está en `.gitignore`. Para ver la última consulta con todo el detalle:

```
python -c "import json,sys; print(json.dumps(json.loads(open(sys.argv[1],
encoding='utf-8').read().splitlines()[-1]), ensure_ascii=False, indent=2))" trazas\2026-11-11.jsonl
```

(Cambiá la fecha por la del archivo que haya en `trazas\`.)

Campo de la traza	Qué tiene
`consulta_usuario`	El texto tal como lo escribiste
`plan`	Lo que entendió el router: pedido, intenciones elegidas y fragmentos del texto que las justifican
`validacion`	Observaciones de la validación del plan y, si hubo, la aclaración pedida
`consultas`	Por cada consulta al grafo: submódulo, nombre, el **Cypher ejecutado**, sus parámetros, las **filas que devolvió Neo4j** y los milisegundos
`evidencia`	Lo que recibió el explicador: es lo único que el modelo puede usar para redactar
`intentos`	Cada texto que escribió el modelo y los problemas que le encontró el verificador
`respuesta`, `origen_respuesta`	El texto final y de dónde salió
`duracion_ms`, `segundos`	Cuánto tardó cada paso (`plan`, `consultas`, `explicacion`) y en total
`error`	El detalle técnico si algo falló

Con esto se contestan las cinco preguntas de trazabilidad de la consigna (§3.7): la **regla** (`[:ACTIVA]` en el grafo y la intención `GRUPO.traza_reglas`), la **fuente** (en `evidencia`, el `origen` de los parámetros y la fuente del precio y de la autonomía), la **evidencia** (`evidencia`), la **consulta** (`consultas`) y la **explicación** (`intentos` y `respuesta`). Más detalle en `ARQUITECTURA.md` §8.

---

8. Errores comunes

Del asistente

Lo que ves	Causa	Qué hacer
`No pude consultar la base de conocimiento (Neo4j) en este momento…` (`origen: error`). Antes pueden salir líneas `Transaction failed and will be retried…`	Neo4j apagado, URI o puerto incorrectos, o contraseña mal cargada	1) Iniciá Neo4j. 2) Corré `GET /salud` y mirá `neo4j.error`. 3) Revisá `NEO4J_URI` y `NEO4J_PASSWORD` **en esta ventana** (sección 4.1). En la traza, `error` empieza con el tipo de excepción: `ServiceUnavailable` es que no hay conexión, `AuthError` que la contraseña no es la correcta. Este error sale **incluso con una pregunta fuera de alcance**, porque el sistema lee el catálogo de Neo4j antes de interpretar la consulta
`No pude usar el modelo de lenguaje (Ollama) en este momento…` (`origen: error`)	Ollama apagado, **modelo sin descargar** o equipo tan lento que pasó el límite de 120 s	Corré `ollama list`: tiene que figurar el modelo de `OLLAMA_MODEL`. Si falta, `ollama pull qwen2.5:3b`. Si Ollama no responde, abrilo o corré `ollama serve`. `GET /salud` muestra `ollama.modelo_disponible`. Si fue por lentitud, volvé a intentar: la segunda vez el modelo ya está cargado
`¿Sobre qué pedido? Indicá el requerimiento (por ejemplo req_01) o fijalo con /pedido…`	**Pedido sin especificar.** El sistema nunca asume `req_01`	Escribí el pedido en la consulta, usá `--pedido req_01` o `/pedido req_01` en la consola, o mandá `requerimiento_id` en la API
`No existe el pedido req_99. Los pedidos cargados son: …`	Ese pedido no está en el grafo	Usá uno de los que lista el mensaje
`No encontré Notebook_Z en el catálogo. Modelos disponibles: …`	El modelo no existe	Usá un modelo de la lista (`Notebook_A`, `Notebook_G`…). Podés escribir «la G»
`¿De qué notebook? Indicá el modelo…` o `Para comparar necesito 2 o 3 modelos…`	La consulta pide algo sobre una notebook y no dice cuál	Nombrá el modelo o los modelos
`No entendí qué querés consultar. Por ejemplo: …`	El router no reconoció ninguna intención	Reformulá con una de las consultas de la sección 5

`La consulta es muy larga…`, `No recibí ninguna consulta…` o `No pude entender la consulta: no tiene palabras…`	Entrada vacía, de más de 1500 caracteres o sin letras	Escribí una sola pregunta, más corta
`Cargar o aclarar un pedido nuevo todavía no está disponible…`	Registrar pedidos nuevos no está habilitado en esta versión	Consultá los pedidos que ya están cargados (`req_01`, `req_02`…)
La respuesta es una lista de datos con `Advertencia:` al final (`origen: plantilla_fija`)	El modelo no logró una redacción verificable	Es correcta, solo menos pulida. Probá reformular
Una respuesta nombra pocas notebooks	El texto del modelo se limita a unas 150 palabras	El detalle completo está en la traza (`evidencia`); pedí una notebook o un tema puntual

De la instalación

Lo que ves	Causa	Qué hacer
`Falta NEO4J_PASSWORD en el entorno (ver .env.example).`	No cargaste el `.env` en esta ventana	Corré el bloque de la sección 4.1
Un traceback largo con `ServiceUnavailable` o `Couldn't connect to localhost:7687` en `cargar_grafo` o `verificar_grafo`	Neo4j apagado, o URI incorrecta	Iniciá Neo4j y revisá `NEO4J_URI`
Un traceback con `AuthError` o `Unauthorized`	La contraseña del `.env` no es la de la base	Corregí `NEO4J_PASSWORD` y volvé a cargar las variables
`verificar_grafo` da menos de `11/11 OK`, o el asistente no encuentra pedidos ni notebooks	El grafo no está cargado, o está a medias	Volvé a correr `python -m scripts.cargar_grafo` y luego `verificar_grafo`
`No module named 'asistente'` (o `scripts`)	Estás en otra carpeta	`cd` a la carpeta raíz del proyecto (la que tiene `requirements.txt`)
`No module named 'fastapi'` (o `neo4j`, `langchain_ollama`)	Falta activar el entorno virtual o instalar las dependencias	`.venv\Scripts\Activate.ps1` y `python -m pip install -r requirements.txt`
Al arrancar la API: `[Errno 10048] error while attempting to bind on address ('127.0.0.1', 8000)… solo se permite un uso de cada dirección de socket`	Hay otro programa usando el puerto 8000	`uvicorn asistente.api:app --port 8001` (y usá ese puerto en las URL)
Las tildes salen rotas (`conexiÃ³n`) al llamar a la API desde PowerShell	Windows PowerShell 5.1 no interpreta bien el JSON en UTF-8 con `Invoke-RestMethod`	Usá la función `Consultar` de la sección 4.3, la consola o `/docs`
Al correr `python -m asistente.cli` con una tubería (`... | python -m ...`) el primer renglón no se entiende	Windows PowerShell 5.1 le agrega una marca invisible (BOM) a lo que se envía por la tubería	No uses tuberías con la consola; escribí en el modo conversación

9. Probar el sistema

No hace falta para usarlo, pero permite comprobar que todo funciona.

```
python -m unittest
```

Sin Neo4j ni Ollama corre las pruebas que no los necesitan y **saltea** el resto (`s` en la salida). Al 11/11/2026 son 171 pruebas, con 26 salteadas. Pueden salir mensajes de conexión rechazada: son de una prueba que simula a propósito un Neo4j caído, no son un error. Con las variables del `.env` cargadas y el grafo en Neo4j, corren también las que consultan la base. Las que usan el modelo se activan aparte, y son lentas:

```
$env:PG3_TEST_LLM = "1"
python -m unittest tests.test_router tests.test_explicador
```

Los resultados de los casos E2E, con la entrada, el plan, el Cypher, la evidencia y la respuesta de cada

uno, van a `docs/pruebas/`.

10. Dónde seguir

Si querés…	Mirá
Entender cómo está armado el sistema	`ARQUITECTURA.md`: componentes, flujo de una consulta y contrato de los submódulos
Saber qué técnicas y herramientas se pueden usar	`RESTRICCIONES.md`
Ver las decisiones del grupo	`INCONSISTENCIAS_CONTEXTOS.md` (D-01 a D-13)
Ver los cambios y arreglos que se hicieron	`CORRECCIONES.md`
Leer el resultado de las pruebas	`pruebas/`
Ver qué se hizo en cada sesión de trabajo	`sesiones/`

11. Linux y macOS

El sistema es el mismo; cambian los comandos de la terminal. Python es `python3` en la mayoría de los sistemas.

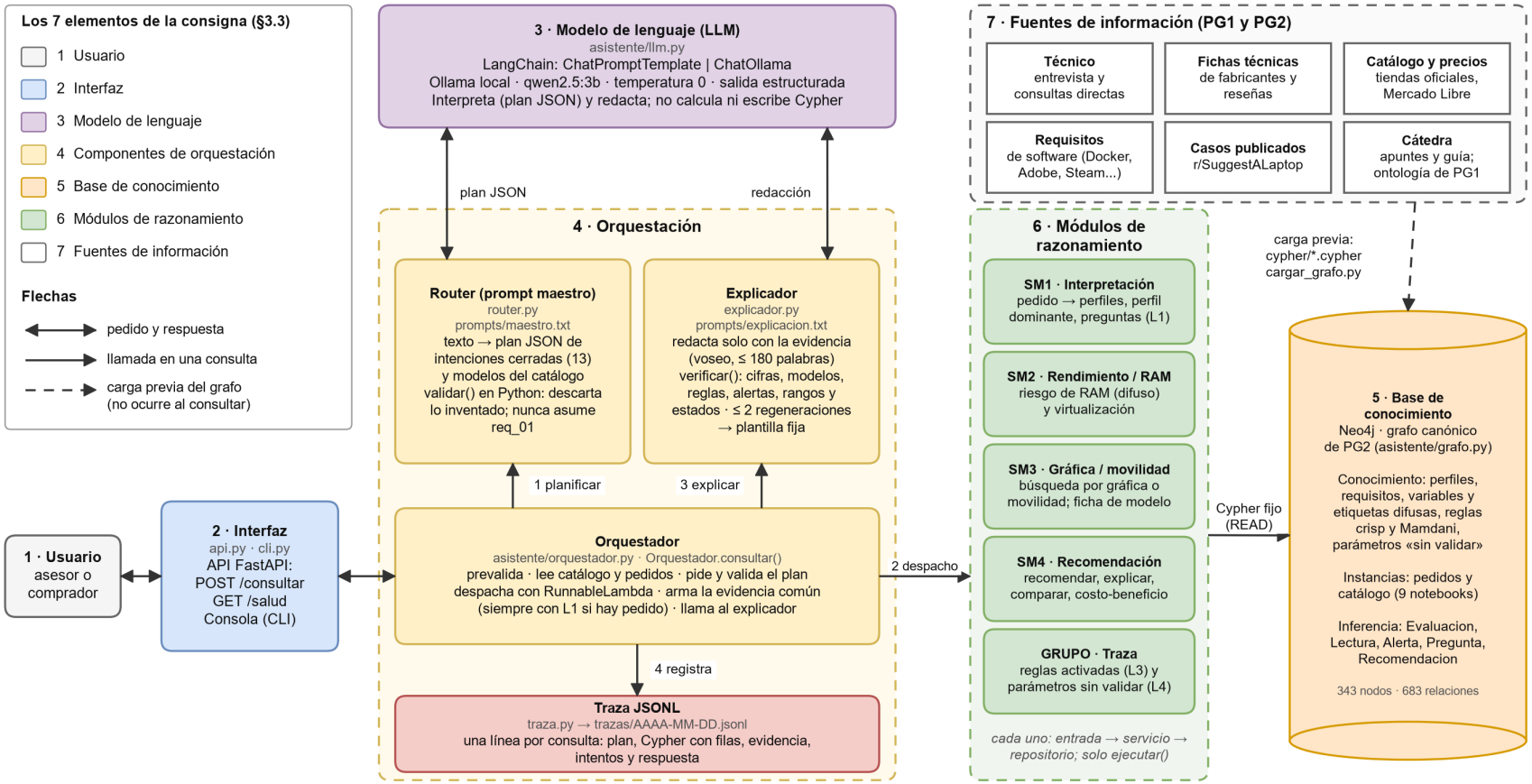
Qué	PowerShell (Windows)	Bash o zsh (Linux, macOS)
Crear y activar el entorno	`python -m venv .venv` y `.venv\Scripts\Activate.ps1`	`python3 -m venv .venv` y `source .venv/bin/activate`
Copiar el `.env`	`Copy-Item .env.example .env`	`cp .env.example .env`
Cargar el `.env` en la terminal	El bloque de la sección 4.1	`set -a; source .env; set +a`
Una variable puntual	`$env:PG3_TEST_LLM = "1"`	`export PG3_TEST_LLM=1`
Consultar la API	La función `Consultar`	`curl -s http://localhost:8000/consultar -H "Content-Type: application/json" -d '{"texto": "Para req_01, ¿qué notebook me recomendás?", "requerimiento_id": "req_01"}'`
Ver la última traza	El comando de la sección 7	`tail -n 1 trazas/AAAA-MM-DD.jsonl | python3 -m json.tool --no-ensure-ascii`

El resto (`python -m scripts.cargar_grafo`, `python -m scripts.verificar_grafo`, `python -m asistente.cli …`, `uvicorn asistente.api:app`, `ollama pull qwen2.5:3b`) es idéntico. Ollama y Neo4j se instalan desde sus sitios para cada sistema.

Anexo C · Diagramas del sistema · 1. Arquitectura

Arquitectura del asistente de selección de notebooks · PG3 · Grupo 2 · UTN FRM

Neo4j maneja la verdad. El LLM interpreta (router) y explica (explicador): no calcula, no decide y no escribe Cypher.



Cómo leer el diagrama

- Neo4j maneja la verdad: cifras, estados, reglas y alertas salen del grafo.
- Las reglas crisp y difusas (Mamdani) se calculan al cargar el grafo (cypher/):
  en cada consulta el asistente solo lee sus resultados (Grafo.leer, READ_ACCESS).
- Las consultas Cypher son plantillas fijas de cada submódulo; el LLM no las escribe.
- Cada submódulo es un paquete entrada → servicio → repositorio (+ esquemas),
  expone solo ejecutar(intencion, parametros, grafo) y no importa a otro submódulo.

Caminos de una consulta (numeración de las flechas)

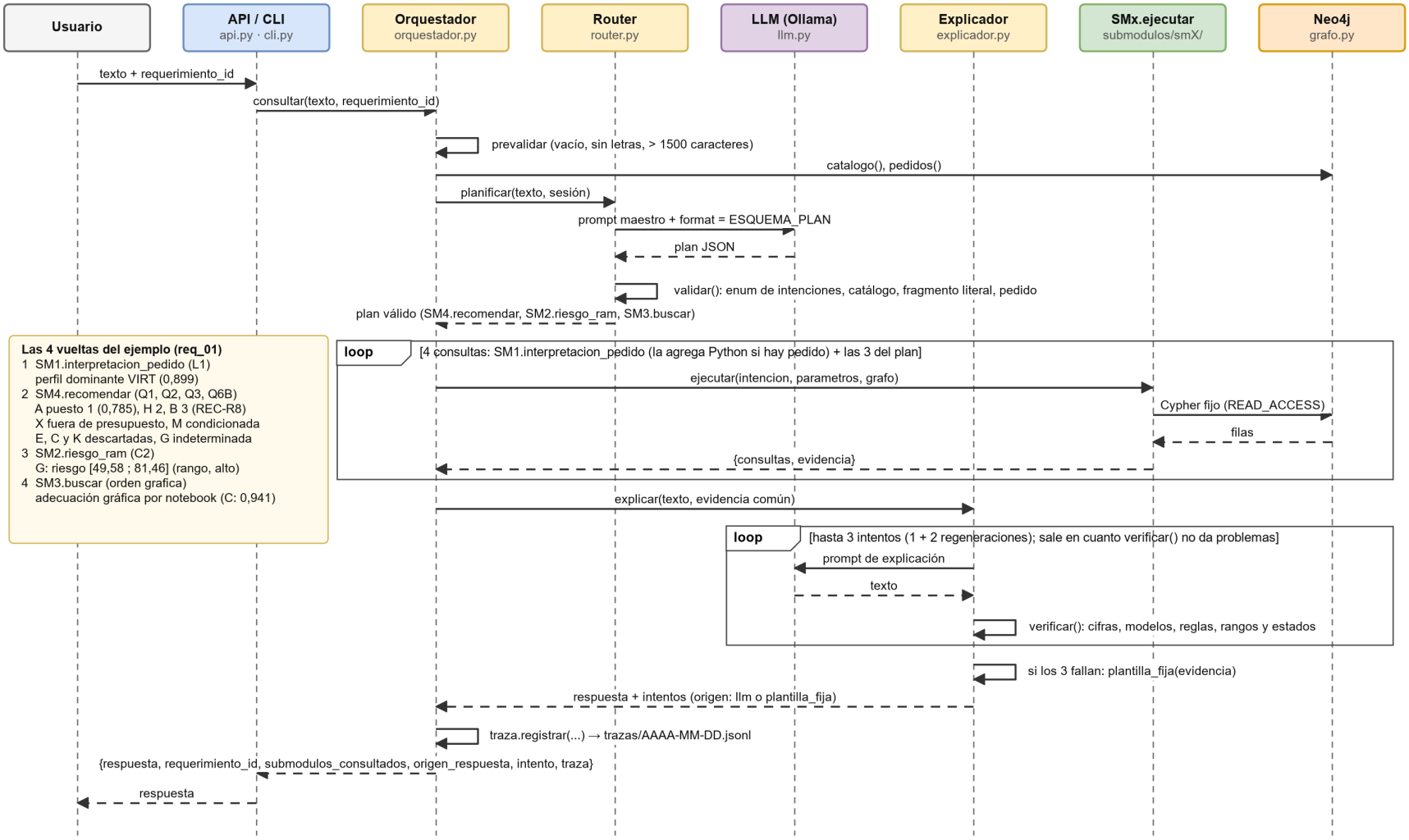
- Camino completo: 1 plan → 2 despacho → 3 explicación → 4 traza.
  origen_respuesta = llm, o plantilla_fija si el verificador rechaza 3 intentos.
- Sin explicador: entrada inválida (validacion), fuera de alcance (mensaje fijo,
  sin tocar el grafo), plan con aclaración, y Neo4j u Ollama caídos (error).
- Las fuentes no se consultan en vivo: se curaron en PG1 y PG2 y entran al grafo
  con cargar_grafo.py. Cada consulta deja su traza (plan, Cypher, filas, respuesta).

Anexo C · Diagramas del sistema · 2. Secuencia de una consulta (E2E-1)

Secuencia de una consulta · ejemplo E2E-1 sobre req_01

Consulta: «Para req_01, ¿qué notebook me recomendás? ¿Cómo andan de RAM y de gráfica?»

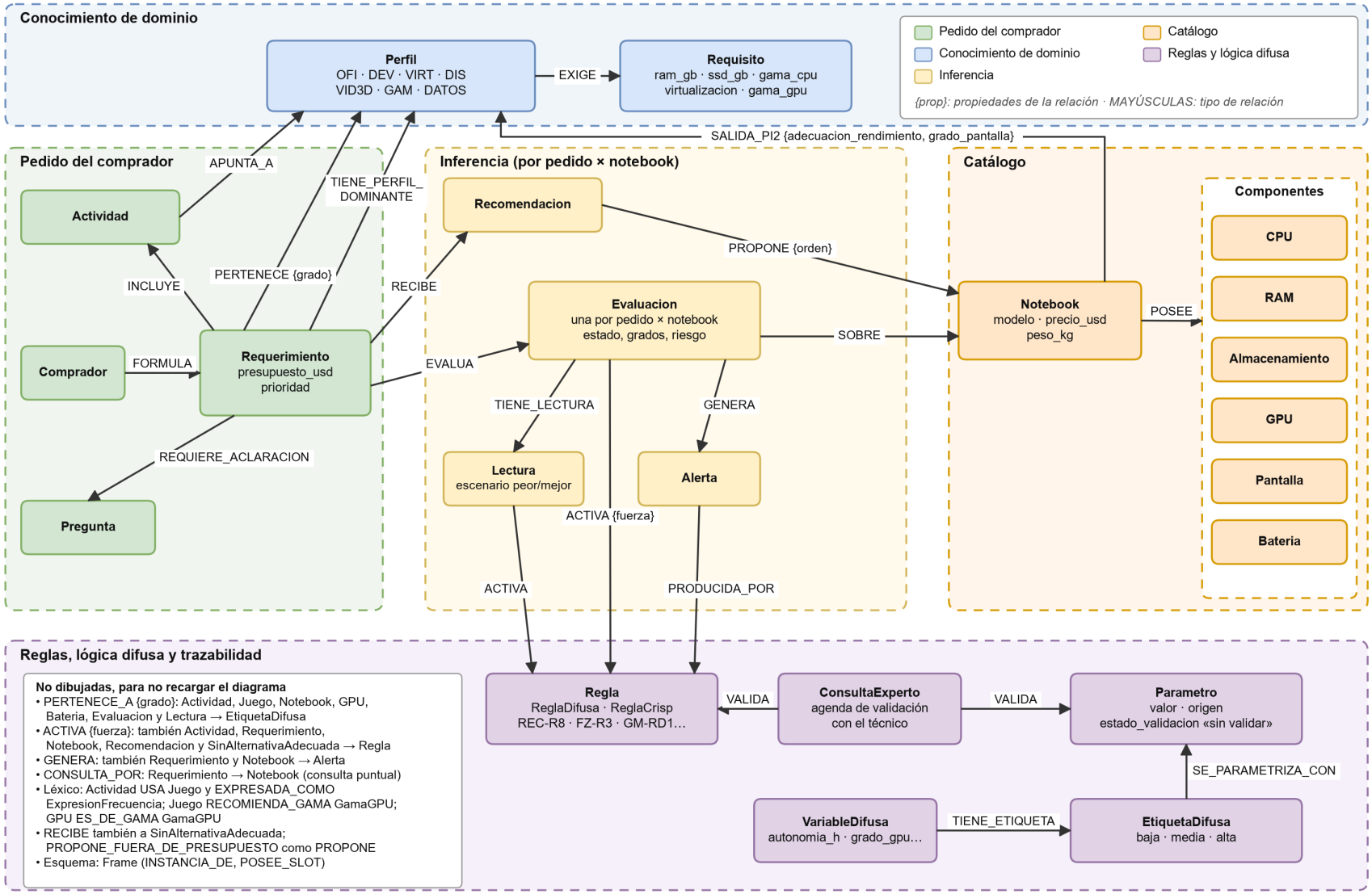
El plan es el esperado para ese caso y los valores salen de tests/fixtures (filas reales del grafo). La redacción del LLM no se muestra.



Anexo C · Diagramas del sistema · 3. Ontología del grafo

Ontología del grafo de conocimiento en Neo4j · PG3 · Grupo 2

Labels principales (cajas) y relaciones (flechas, con sus propiedades entre llaves). Fuente: cypher/*.cypher, partido del canónico de PG2.



Anexo D · Casos de uso integrados (consigna §3.4)

Cada caso involucra **al menos dos submódulos distintos** además de SM1, que el orquestador agrega siempre que hay un pedido. Los datos salen de las corridas reales de `scripts/correr_e2e.py`; el detalle de cada una está en `E2E-n.md`.

Caso 1 · E2E-1

Consulta: «Para req_01, ¿qué notebook me recomendás? ¿Cómo andan de RAM y de gráfica?»

Situación. Un asesor atiende el pedido req_01 (presupuesto 1000 USD, perfil de virtualización) y quiere una recomendación, con el riesgo de RAM y la gráfica de las candidatas.

Información usada. Submódulos del plan: SM2, SM4 (más SM1 para el pedido). Intenciones: `SM4.recomendar`, `SM2.riesgo_ram`. Consultas Cypher ejecutadas: 6.

Razonamiento. SM4 ordena con REC-R8 (grado de recomendación y desempate por ampliabilidad de la RAM); SM2 calcula el riesgo de RAM con sus reglas difusas; SM3 aporta la gráfica y la movilidad. Las excluyentes (REC-R3) descartan a E y a C, y el dato faltante (REC-R15) deja a M condicionada.

Resultado. Al menos dos submódulos distintos pedidos en el plan (sin L1) → sí; Ranking: A puesto 1, H puesto 2, B puesto 3 → sí; Grado de A = 0,785 → sí; X queda fuera de presupuesto → sí; M condicionada, E y C descartadas, G indeterminada → sí.

Explicación generada.

> Te recomiendo la Notebook_A. La RAM es mixta y la gráfica está bien equipada. La Notebook_H también está bien, pero la RAM es mixta y la gráfica es de baja. Los parametros difusos y los umbrales del sistema estan sin validar por el tecnico: los grados y riesgos son una referencia, no una certeza.

Origen: llm · 7.091 s · detalle

Caso 2 · E2E-2

Consulta: «En req_01, ¿por qué la M quedó condicionada y la G indeterminada? ¿Qué riesgo de RAM tiene la G?»

Situación. El asesor ve en la lista a la M «condicionada» y a la G «indeterminada» y no entiende por qué.

Información usada. Submódulos del plan: SM2, SM4 (más SM1 para el pedido). Intenciones: `SM4.explicar_notebook`, `SM4.explicar_notebook`, `SM2.riesgo_ram`. Consultas Cypher ejecutadas: 4.

Razonamiento. M no informa la autonomía: REC-R15 deja su grado como rango en vez de inventar un valor. G no informa si la CPU soporta virtualización ni si la RAM es ampliable: el grafo no puede decidir y el estado es indeterminado. SM2 informa el riesgo de RAM de G como rango, por la misma falta de dato.

Resultado. Al menos dos submódulos distintos pedidos en el plan (sin L1) → sí; M condicionada, grado [0,504 ; 0,826], falta la autonomía → sí; G indeterminada con las alertas REN-R10 y FZ-R7 → sí; Riesgo de RAM de G como rango [49,58 ; 81,46] (SM2) → sí.

Explicación generada.

> La M quedó condicionada porque falta la información sobre la autonomía, mientras que la G es indeterminada porque no informa si la RAM es ampliable. La G tiene un riesgo de RAM del 52.9%, con un riesgo alto de quedarse corta de RAM. Los parametros difusos y los umbrales del sistema estan sin validar por el tecnico: los grados y riesgos son una referencia, no una certeza.

Origen: llm · 5.087 s · detalle

Caso 3 · E2E-5

Consulta: «En req_01, la E tiene casi el mismo puntaje que la B y la H, ¿por qué no la recomiendan? ¿Qué pasa con la virtualización?»

Situación. El asesor ve que la E puntúa igual que la B y la H y se pregunta por qué no entra en la recomendación.

Información usada. Submódulos del plan: SM2, SM4 (más SM1 para el pedido). Intenciones: `SM4.comparar`, `SM2.virtualizacion`. Consultas Cypher ejecutadas: 5.

Razonamiento. El puntaje no alcanza: la virtualización es un requisito excluyente del perfil (REC-R3) y la CPU de la E no la soporta, así que queda descartada aunque su adecuación global sea ≈ 0,81. SM2 confirma la virtualización por hardware.

Resultado. Al menos dos submódulos distintos pedidos en el plan (sin L1) → sí; E ≈ 0,81, B = 0,813 y H = 0,811 → sí; E Descartada (excluyente incumplido) por REC-R3 → sí; B y H siguen recomendadas → sí.

Explicación generada.

> La B y la H cumplen con la virtualización, pero la E no soporta virtualización por hardware, cumpliendo con la alerta excluyente. La recomendación está desempatada por la ampliabilidad, la E es la única descartada. Los parametros difusos y los umbrales del sistema estan sin validar por el tecnico: los grados y riesgos son una referencia, no una certeza.

Origen: llm · 4.515 s · detalle

Caso 4 · E2E-9

Consulta: «En req_01, ¿qué reglas se activaron para la A, de dónde sale cada dato y qué riesgo de RAM tiene?»

Situación. Un auditor pide reconstruir cómo llegó el sistema al resultado de la A: reglas, fuente de los datos y riesgo de RAM (consigna §3.7).

Información usada. Submódulos del plan: GRUPO, SM2 (más SM1 para el pedido). Intenciones: `GRUPO.traza_reglas`, `SM2.riesgo_ram`. Consultas Cypher ejecutadas: 6.

Razonamiento. GRUPO.traza_reglas devuelve las 18 reglas activadas de los cuatro submódulos con su fuerza y, para 14 de ellas, los parámetros que las respaldan. Las 4 restantes salen «no registrada»: el grafo no guarda su fuente. SM2 informa el riesgo de RAM de la A.

Resultado. Al menos dos submódulos distintos pedidos en el plan (sin L1) → sí; Traza de A: reglas de los 4 submódulos, 18 reglas, 14 con origen y 4 «no registrada» → sí; La notebook trae la fuente del precio y de la autonomía → sí; Riesgo de RAM de A, tal cual lo da el grafo (SM2) → sí.

Explicación generada.

> Para la A, se activaron las reglas FZ-R2, FZ-R4 y FZ-R5. Los datos vienen de la evaluación del grupo (PI3). La RAM tiene un riesgo de 23.02, que es bajo. Los parametros difusos y los umbrales del sistema estan sin validar por el tecnico: los grados y riesgos son una referencia, no una certeza.

Origen: llm · 4.332 s · detalle

Caso 5 · E2E-10

Consulta: «Para req_01, ¿cuáles son las mejores en movilidad y cómo queda el costo-beneficio?»

Situación. El asesor quiere las más portables del pedido y cuál rinde más por su precio.

Información usada. Submódulos del plan: SM3, SM4 (más SM1 para el pedido). Intenciones: `SM4.costo_beneficio`, `SM3.buscar`. Consultas Cypher ejecutadas: 3.

Razonamiento. SM3 ordena por la adecuación de movilidad (peso y autonomía) y deja los datos faltantes al final; SM4 ordena por el índice costo-beneficio de REC-R9 (adecuación global sobre precio).

Resultado. Al menos dos submódulos distintos pedidos en el plan (sin L1) → sí; SM3: orden por movilidad descendente, con los datos faltantes al final y sin números inventados → sí; SM4: costo-beneficio de Q7 (B 1,161; A 0,907) → sí.

Explicación generada.

> Vos te conviene la Notebook_A, está recomendada (Alta) en movilidad. El costo-beneficio está en 0.907. Los parametros difusos y los umbrales del sistema estan sin validar por el tecnico: los grados y riesgos son una referencia, no una certeza.

Origen: llm · 4.281 s · detalle

Controles

No cuentan como casos integrales: no cruzan información entre submódulos.

- **E2E-3** · «Para req_02, ¿qué notebook me conviene?» → APRUEBA. El asesor pregunta por req_02, un pedido al que todavía le falta una respuesta del comprador. El perfil de req_02 es indeterminado y no tiene evaluaciones: el sistema no completa un perfil (no asume oficina), pide responder la pregunta pendiente y no recomienda. (detalle)
- **E2E-8** · «¿Cuál es la capital de Francia?» → APRUEBA. Una pregunta que no tiene que ver con notebooks. El router la marca fuera de alcance y el sistema responde con un mensaje fijo, sin consultar el grafo ni el explicador. (detalle)

Anexo E · Pruebas funcionales (consigna §3.5)

Generado el 2026-11-11 sobre Neo4j 5.26 con el grafo cargado desde `cypher/`. Corrida completa: `python -m unittest` (171 tests, 2 salteados que piden Ollama) y `python -m scripts.verificar_grafo` (CP1 a CP11, 11/11).

Recuperación · tests/test_sm3.py · `test_x_grado_gpu_074` · sí

- **Qué verifica:** SM3.buscar con el filtro `dedicada`: X con grado de GPU 0,74
- **Esperado:** Solo X (y las dedicadas) con 0,74
- **Obtenido (corrida):** pasa

Recuperación · tests/test_sm3.py · `test_ficha_m` · sí

- **Qué verifica:** SM3.ficha de la M: la autonomía desconocida viaja como `null`, no como un número
- **Esperado:** autonomía `null`, tipo `desconocida`
- **Obtenido (corrida):** pasa

Recuperación · tests/test_sm4.py · `test_los_pedidos_no_se_mezclan` · sí

- **Qué verifica:** SM4 consulta solo las evaluaciones del pedido pedido (los pedidos no se mezclan)
- **Esperado:** Ningún dato de otro pedido
- **Obtenido (corrida):** pasa

Consulta · tests/test_sm4.py · `test_solo_lectura`; tests/test_grafo.py · sí

- **Qué verifica:** Cada intención ejecuta su plantilla Cypher fija y de solo lectura
- **Esperado:** Sin MERGE/CREATE/SET/DELETE; `READ_ACCESS`
- **Obtenido (corrida):** pasa

Consulta · tests/test_sm2.py · `test_virtualizacion_coincide_con_c1` · sí

- **Qué verifica:** SM2.virtualizacion coincide con la consulta C1 del canónico
- **Esperado:** Mismas notebooks descartadas
- **Obtenido (corrida):** pasa

Consulta · scripts/contar_grafo.py · docs/pruebas/conteos_grafo.md · sí

- **Qué verifica:** Los conteos del grafo dividido igualan al canónico
- **Esperado:** 343 nodos y 683 relaciones
- **Obtenido (corrida):** coincide

Regla · scripts/verificar_grafo.py · CP2; tests/test_sm4.py · `test_e2e_5_el_excluyente_gana_al_puntaje` · sí

- **Qué verifica:** REC-R3: E sin virtualización queda descartada aunque puntúe 0,81
- **Esperado:** `Descartada (excluyente incumplido)`
- **Obtenido (corrida):** 11/11 CP

Regla · scripts/verificar_grafo.py · CP7 · sí

- **Qué verifica:** REC-R8: desempate por ampliabilidad (A 1, H 2, B 3)
- **Esperado:** A, H, B
- **Obtenido (corrida):** OK

Regla · tests/test_grupo.py · `test_traza_a`, `test_origen_de_la_traza_de_a` · sí

- **Qué verifica:** La traza de A trae reglas de los 4 submódulos, con su origen cuando el grafo lo tiene
- **Esperado:** 18 reglas, 14 con origen, 4 «no registrada»
- **Obtenido (corrida):** pasa

Difuso · scripts/verificar_grafo.py · CP4; tests/test_sm2.py · sí

- **Qué verifica:** G indeterminada: riesgo de RAM como rango [49,58 ; 81,46] y alertas REN-R10 y FZ-R7
- **Esperado:** Rango sin promediar

- **Obtenido (corrida):** OK

Difuso · scripts/verificar_grafo.py · CP9; tests/test_sm4.py · `test_e2e_2_m_rango_y_falta_la_autonomia` · SÍ

- **Qué verifica:** M sin autonomía: condicionada, grado [0,504 ; 0,826]
- **Esperado:** Rango, no un número
- **Obtenido (corrida):** OK

Difuso · scripts/verificar_grafo.py · CP8 · sí

- **Qué verifica:** 47 etiquetas difusas sin validar en 15 variables
- **Esperado:** 47 y 15
- **Obtenido (corrida):** OK

Respuesta · tests/test_sm4.py · `test_literales_d06_en_todo_lo_que_devuelve` · SÍ

- **Qué verifica:** Los literales de estado y validación son los de D-06 en todo lo que devuelve SM4
- **Esperado:** Sin tildes, `sin validar`
- **Obtenido (corrida):** pasa

Respuesta · tests/test_explicador.py (≈50 casos, uno por control a–h) · sí

- **Qué verifica:** El verificador rechaza cifras, modelos, reglas y alertas sin respaldo
- **Esperado:** Cada control se dispara
- **Obtenido (corrida):** pasa

Respuesta · tests/test_explicador.py · `TestExplicar`, `TestPlantillaFija` · SÍ

- **Qué verifica:** Si el LLM falla dos veces, sale la plantilla fija, que pasa su propio verificador
- **Esperado:** `origen_respuesta = plantilla_fija`
- **Obtenido (corrida):** pasa

Respuesta · scripts/correr_e2e.py · docs/pruebas/resultados_e2e.md · sí

- **Qué verifica:** E2E-1, 2, 3, 5, 8, 9 y 10 contra el orquestador real
- **Esperado:** Los esperados salen del grafo
- **Obtenido (corrida):** 7 de 7

Cubre los cinco tipos de §3.5: recuperación, consulta, regla, difuso y respuesta. Los E2E (casos que cruzan submódulos) están en `resultados_e2e.md` y `casos_de_uso.md`.